

Wymagający złamania wierszy tytuł pracy w języku polskim

(English title)

Maksymilian Debeściak

Praca licencjacka

Promotor: dr Jan Kowalski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

16 maja 2026

Streszczenie

...



...

Spis treści

1. Wprowadzenie	7
1.1. Dedukcja naturalna	7
1.2. Zapis tekstowy dowodów	9
2. Reprezentacja formuł, sekwentów, reguł wnioskowania i dowodów	15
2.1. Reprezentacja formuł i sekwentów	15
2.2. Reprezentacja reguł wnioskowania	16
2.3. Reprezentacja dowodów	17
2.3.1. Modele językowe	20
3. Strategie konstruowania dowodów	23
3.1. Motywacja	23
3.2. Strategia pierwsza: liniowe dopisywanie kroków dowodu	24
3.2.1. Idea strategii	24
3.2.2. Generowanie i weryfikacja pojedynczej linii	25
3.2.3. Warunki zakończenia	27
3.2.4. Ograniczenia strategii	27
3.3. Strategia druga: przeszukiwanie wielu częściowych dowodów	28
3.3.1. Motywacja	28
3.3.2. Zbiór częściowych dowodów	28
3.3.3. Ocena prawdopodobieństwa częściowego dowodu	29
3.3.4. Pełny przebieg strategii	30
3.3.5. Ograniczenia strategii	31

3.4. Strategia trzecia: dekompozycja celu i drzewo poszukiwania	32
3.4.1. Motywacja	32
3.4.2. Rozbicie problemu na podproblemy	33
3.4.3. Format zapytania o rozbicie	34
3.4.4. Sprawdzanie poprawności rozbicia	35
3.4.5. Generowanie rozbić przez model	37
3.4.6. Dowodzenie sekwentów z założeniami	38
3.4.7. Drzewo poszukiwań	42
3.4.8. Sklejanie dowodów	47
4. Generowanie korpusu	51
4.1. Motywacja	51
4.2. Formuły źródłowe i początkowe dowody	52
4.3. Losowe rozszerzanie dużego dowodu	52
4.4. Generowanie z dużego dowodu przykładów treningowych	55
4.4.1. Wyłuskiwanie dowodów tautologii	56
4.4.2. Tworzenie dowodów z założeniami	57
4.4.3. Tworzenie rozbić sekwentów	59
4.5. Losowe rozszerzanie dużego dowodu	64
4.6. Wyłuskiwanie dowodów tautologii	64
4.7. Normalizacja formuł i usuwanie powtórzeń	64
4.8. Generowanie problemów pomocniczych	64
4.9. Generowanie smashy	64
4.10. Normalizacja kontekstów w smashach	64
4.11. Równoważność jako przypadek specjalny	64
4.12. Walidacja i czyszczenie korpusu	64
4.13. Podsumowanie	64
5. Trenowanie i kożystanie z modelu (NanoGPTBundle i te wszystkie funkcje z nim związane !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!, podkreślam WSZYSTKIE(liczenie prawdopodobieństwa, dodawanie tokenu, linii, trenowanie, dotrenowanie, ładowanie)!!!!)	65

Rozdział 1.

Wprowadzenie

Dedukcja naturalna jest metodą dowodzenia twierdzeń logicznych przez wychodzenie z pewnych założeń i modyfikowanie ich za pomocą ściśle zdefiniowanych reguł wnioskowania.

1.1. Dedukcja naturalna

Definicja 1. Sekwentem nazywamy wyrażenie postaci

$$\Gamma \vdash \varphi$$

gdzie Γ jest skończonym zbiorem formuł (zapisywanych po przecinku i bez nawiasów klamrowych) zwanym *kontekstem*, a φ jest formułą zwaną *tezą sekwentu*. Sekwent $\Gamma \vdash \varphi$ interpretujemy jako stwierdzenie, że jeśli założymy, że wszystkie formuły z Γ są prawdziwe, da się dowieść φ . W rachunku zdań w logice klasycznej, który będzie jedyną logiką w której się będziemy poruszać, zdanie prawdziwe we wszystkich wartościowaniach jest dowodliwe, tak więc $\Gamma \vdash \phi$ jest równoważne stwierdzeniu jeżeli wszystkie formuły kontekstu Γ są prawdziwe to ϕ jest prawdziwe.

Definicja 2. Regułą wnioskowania nazywamy schemat postaci

$$\frac{S_1 \quad S_2 \quad \dots \quad S_n}{S}(R)$$

gdzie $S_1, \dots, S_n, S$ są sekwentami. Sekwenty $S_1, \dots, S_n$ nazywamy *przesłankami reguły*, natomiast sekwent S nazywamy *konkluzją reguły*. R to z kolei *indeks reguły*, który służy do jej identyfikacji. Jeżeli przesłanki reguły są prawdziwe to z definicji jej konkluzja jest również prawdziwa.

Dowody w dedukcji naturalnej można utożsamiać z drzewami, których węzłami są sekwenty, a krawędzie są indeksowane indeksami reguł wnioskowania.

W niniejszej pracy skupimy się na dowodzeniu twierdzeń w logice klasycznej z następującymi spójnikami logicznymi: koniunkcją ($\wedge$), alternatywą ($\vee$), negacją ($\neg$) oraz implikacją ($\rightarrow$). regułami wnioskowania:

$\frac{}{\varphi \vdash \varphi} (\text{Ass})$	$\frac{\Gamma \vdash \varphi}{\Gamma, \psi \vdash \varphi} (\text{W})$
$\frac{\Gamma, \varphi \vdash \psi}{\Gamma \vdash \varphi \rightarrow \psi} (\rightarrow I)$	$\frac{\Gamma_1 \vdash \varphi \rightarrow \psi \quad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \psi} (\rightarrow E)$
$\frac{\Gamma, \varphi \vdash \perp}{\Gamma \vdash \neg \varphi} (\neg I)$	$\frac{\Gamma_1 \vdash \neg \varphi \quad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \perp} (\neg E)$
$\frac{\Gamma_1 \vdash \varphi \quad \Gamma_2 \vdash \psi}{\Gamma_1, \Gamma_2 \vdash \varphi \wedge \psi} (\wedge I)$	$\frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \varphi} (\wedge E_1)$
$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi} (\vee I_1)$	$\frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \psi} (\wedge E_2)$
$\frac{\Gamma \vdash \psi}{\Gamma \vdash \varphi \vee \psi} (\vee I_2)$	$\frac{\Gamma_1 \vdash \varphi \vee \psi \quad \Gamma_2, \varphi \vdash \rho \quad \Gamma_3, \psi \vdash \rho}{\Gamma_1, \Gamma_2, \Gamma_3 \vdash \rho} (\vee E)$
$\frac{}{\vdash \top} (\top I)$	$\frac{\Gamma \vdash \perp}{\Gamma \vdash \varphi} (\perp E)$
$\frac{\Gamma_1, \varphi \vdash \psi \quad \Gamma_2, \psi \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \varphi \leftrightarrow \psi} (\leftrightarrow I)$	$\frac{\Gamma_1 \vdash \varphi \leftrightarrow \psi \quad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \psi} (\leftrightarrow E_1)$
$\frac{\Gamma_1 \vdash \varphi \leftrightarrow \psi \quad \Gamma_2 \vdash \psi}{\Gamma_1, \Gamma_2 \vdash \varphi} (\leftrightarrow E_2)$	$\frac{\Gamma, \neg \varphi \vdash \perp}{\Gamma \vdash \varphi} (\text{RAA})$
$\frac{\Gamma \vdash \neg \neg \varphi}{\Gamma \vdash \varphi} (\neg \neg)$	$\frac{}{\vdash \varphi \vee \neg \varphi} (\text{TND})$

Tabela 1.1: Reguły wnioskowania rozważanego systemu

Oto przykładowy dowód w dedukcji naturalnej w notacji drzewiastej:

$$\frac{\frac{\frac{}{a \vee b \vdash a \vee b} (\text{Ass}) \quad \frac{\frac{}{a \vdash a} (\text{Ass})}{a \vdash b \vee a} (\vee I_2)}{a \vee b \vdash b \vee a} (\vee E) \quad \frac{}{b \vdash b} (\text{Ass})}{b \vdash b \vee a} (\vee I_1)}{\vdash (a \vee b) \rightarrow (b \vee a)} (\rightarrow I)$$

Dużą zaletą dedukcji naturalnej jest to, że niejako symuluje ona sposób, w jaki ludzie przeprowadzają wnioskowanie, zarówno w matematyce, jak i w codziennym życiu. Dowody mają swoją drzewiastą strukturę, którą można łatwo analizować. Inną zaletą tej metody jest jej uniwersalność. Dodając odpowiednie reguły wnioskowania możemy otrzymać dedukcję naturalną dla różnych systemów logicznych np logiki pierwszego i drugiego rzędu, czy logiki modalnej. Jeszcze jedną zaletą jest to, że w przypadku większości użytecznych dowodów jesteśmy w stanie uniknąć wykładniczo dużej złożoności wielu innych systemów dowodzenia sprowadzających się

do rozpatrzenia wszystkich możliwych przypadków. Z drugiej strony dedukcja naturalna jest trudna w zaimplementowaniu. Zwykle dowód przeprowadzamy z góry na dół, zaczynając od założeń i kończąc na tezie. Jednak nie zawsze jasne jest, którą regułę wnioskowania zastosować i co gorsza wyniki zastosowania niektórych reguł mogą zawierać podformuły których nie ma w przesłankach, lub jej wybór nie jest jednoznaczny. Np. używając reguły

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi} (\vee I_1)$$

w przesłance nie mamy informacji o formule ψ , która pojawi się w konkluzji. Jednak podobieństwo dedukcji naturalnej do ludzkiego sposobu wnioskowania sprawia, oraz pewien dający się zauważyć porządek w zapisie dowodów sprawia, że gdzie klasyczna implementacja dedukcji naturalnej jest trudna, wydaje się ona dobrze nadawać do implementacji z użyciem metod sztucznej inteligencji, w tym modeli językowych o architekturze transformerów.

W niniejszej pracy przedstawimy implementację automatycznego systemu dowodzenia twierdzeń w logice klasycznej przy użyciu odpowiednio wytrenowanego modelu nano-GPT oraz przedstawimy wyniki jego działania i jak dobrze radzi sobie z przeprowadzaniem dowodów. Konstrukcja systemu będzie opisana krok po kroku, zaczynając od zaprogramowania logicznej struktury systemu, tłumaczenia zapisu dowodu w formie tekstowej na strukturę logiczną i sprawdzanie poprawności dowodu, generacja korpusu treningowego, trenowanie modelu oraz kolejne etapy konstrukcji samego systemu dowodzenia (wybór możliwych linii, równoległość prowadzenia kilku ścieżek rozumowania, itp.).

1.2. Zapis tekstowy dowodów

Notacja Fitcha jest sposobem zapisu dowodów dedukcji naturalnej w formie tekstowej. Jest on wygodny, ponieważ metoda zapisywania drzew dowodu potrafi zajmować bardzo dużo miejsca i trudno użyć do niej modeli językowych.

Z punktu widzenia implementacji komputerowej klasyczna notacja Fitcha narzuca ograniczenia wynikające ze struktury zagnieżdżonych bloków, które określają, do których założeń można się odwoływać w danym miejscu dowodu. W konsekwencji dostęp do wcześniejszych kroków dowodu zależy od ich położenia w tej strukturze.

W stosowanej w niniejszej pracy notacji dowód koduje uporządkowaną listę sekwentów, w której każda linia koduje jeden sekwent, a zależności między kolejnymi krokami są wyrażane poprzez odwołania do indeksów wcześniejszych linii oraz zastosowane reguły wnioskowania. Nie występują tu ograniczenia wynikające ze zagnieżdżonych bloków, a dowolny wcześniejszy sekwent może zostać użyty jako przesłanka, o ile spełnia warunki danej reguły.

Takie podejście pozwala traktować konstrukcję dowodu jako proces składania sekwentów na podstawie wcześniej uzyskanych wyników, bez konieczności odtwarzania struktury zagnieżdżeń charakterystycznej dla klasycznej notacji Fitcha.

Co więcej, w stosowanej notacji każdej linii zapisywana jest jawnie teza sekwentu, który dana linia koduje. Jest to użyteczne podczas sprawdzania poprawności dowodu.

Podejście to wprowadza jednak pewną niedogodność: nie istnieje jedna reguła, pozwalająca wyciągnąć formułę należącą do kontekstu sekwentu w jednym kroku. Ponieważ jest to dość częsta operacja wprowadzamy dwie pomocnicze reguły, których celem nie jest wprowadzanie nowych własności logicznych systemu, lecz stanowią wygodny mechanizm umożliwiający prostsze manipulowanie kontekstem.

$$\frac{\varphi, \Gamma \vdash \psi}{\Gamma, \varphi \vdash \varphi} \text{ (FromContext)} \quad \frac{\Gamma \vdash \psi}{\Gamma, \varphi \vdash \varphi} \text{ (FromWeakenContext)}$$

Poniżej przedstawiam drzewa dowodu poprawności tych reguł:

$$\frac{\frac{\Gamma, \varphi \vdash \psi \quad \frac{}{\varphi \vdash \varphi} \text{ (Ass)}}{\Gamma, \varphi \vdash \psi \wedge \varphi} (\wedge I)}{\Gamma, \varphi \vdash \varphi} (\wedge E_2)$$

Dowód reguły (FromContext)

$$\frac{\frac{\Gamma \vdash \psi \quad \frac{}{\varphi \vdash \varphi} \text{ (Ass)}}{\Gamma, \varphi \vdash \psi \wedge \varphi} (\wedge I)}{\Gamma, \varphi \vdash \varphi} (\wedge E_2)$$

Dowód reguły (FromWeakenContext)

Aby sformalizować podejście, w którym każda linia dowodu koduje jeden sekwent, wprowadzamy oznaczenia pozwalające jednoznacznie interpretować poszczególne linie dowodu oraz odwołania między nimi:

`Fitch(i)` := sekwent kodowany przez linię o indeksie `i`

oraz oznaczenie

`s.conclusion` := teza sekwentu `s`

`s.assumptions` := kontekst sekwentu `s`

Poniżej przedstawimy interpretację poszczególnych reguł wnioskowania w notacji Fitcha, a także ich odpowiedniki w notacji drzewiastej:

Notacja Fitcha	Notacja drzewiasta
i. ϕ assumption	$\overline{\phi \vdash \phi} \text{ (Ass)}$
j. ψ k. ϕ i. ϕ weakening, j, k	$\frac{\text{Fitch}(k)}{\text{Fitch}(i)} (W)$ gdzie $\text{Fitch}(i).\text{assumptions} =$ $\text{Fitch}(k).\text{assumptions},$ $\text{Fitch}(j).\text{conclusion}$
j. ϕ k. ψ i. $\phi \rightarrow \psi$ $\rightarrow$ -introduction, j-k	$\frac{\text{Fitch}(k)}{\text{Fitch}(i)} (\rightarrow I)$
j. ϕ k. $\perp$ i. $\neg\phi$ $\neg$ -introduction, j-k	$\frac{\text{Fitch}(k)}{\text{Fitch}(i)} (\neg I)$
j. ϕ k. $\phi \rightarrow \psi$ i. ψ $\rightarrow$ -elimination, k, j	$\frac{\text{Fitch}(k) \quad \text{Fitch}(j)}{\text{Fitch}(i)} (\rightarrow E)$
j. $\neg\phi$ k. ϕ i. $\perp$ $\neg$ -elimination, j, k	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k)}{\text{Fitch}(i)} (\neg E)$
j. ϕ k. ψ i. $\phi \wedge \psi$ $\wedge$ -introduction, j, k	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k)}{\text{Fitch}(i)} (\wedge I)$
j. ϕ i. $\phi \vee \psi$ $\vee$ -introduction1, j	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\vee I_1)$

j. $\psi \quad \dots$ $\dots$ i. $\phi \vee \psi \quad \vee\text{-introduction2, } j$	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\vee I_2)$
i. $\top \quad \top\text{-introduction}$	$\overline{\vdash \top} (\top I)$
j. $\phi \wedge \psi \quad \dots$ $\dots$ i. $\phi \quad \wedge\text{-elimination1, } j$	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\wedge E_1)$
j. $\phi \wedge \psi \quad \dots$ $\dots$ i. $\psi \quad \wedge\text{-elimination2, } j$	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\wedge E_2)$
j. $\phi \vee \psi \quad \dots$ $\dots$ k. $\rho \quad \dots$ $\dots$ l. $\rho \quad \dots$ $\dots$ i. $\rho \quad \vee\text{-elimination, } j, k, l$	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k) \quad \text{Fitch}(l)}{\text{Fitch}(i)} (\vee E)$
j. $\perp \quad \dots$ $\dots$ i. $\phi \quad \perp\text{-elimination, } j$	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\perp E)$
j. $\psi \quad \dots$ $\dots$ k. $\phi \quad \dots$ $\dots$ i. $\phi \leftrightarrow \psi \quad \leftrightarrow\text{-introduction, } j, k$	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k)}{\text{Fitch}(i)} (\leftrightarrow I)$
j. $\phi \leftrightarrow \psi \quad \dots$ $\dots$ k. $\phi \quad \dots$ $\dots$ i. $\psi \quad \leftrightarrow\text{-elimination1, } j, k$	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k)}{\text{Fitch}(i)} (\leftrightarrow E_1)$
j. $\phi \leftrightarrow \psi \quad \dots$ $\dots$ k. $\psi \quad \dots$ $\dots$ i. $\phi \quad \leftrightarrow\text{-elimination2, } j, k$	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k)}{\text{Fitch}(i)} (\leftrightarrow E_2)$

j. $\neg\phi$ k. $\perp$ i. ϕ RAA, j-k	$\frac{\text{Fitch}(k)}{\text{Fitch}(i)} \text{ (RAA)}$
j. $\neg\neg\phi$ i. ϕ $\neg\neg$ -elimination, j	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\neg\neg)$
i. $\phi \vee \neg\phi$ TND	$\overline{\vdash \phi \vee \neg\phi} \text{ (TND)}$
j. ψ i. ϕ context, j	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} \text{ (FromContext)}$
j. ψ i. ϕ contextWeak, j	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} \text{ (FromWeakenContext)}$

Warto zauważyć, że aby dowody były poprawne, muszą być spełnione odpowiednie warunki dotyczące sekwentów, do których się odwołujemy, wynikające z postaci reguł wnioskowania. Warunki te nie są jednak podane explicite, lecz wynikają z możliwości dopasowania struktury sekwentów do schematów reguł.

Na przykład dla zapisu:

j. $\phi \vee \psi$...
...
k. ρ ...
...
l. ρ ...
...
i. ρ $\vee$ -elimination, j, k, l

który odpowiada dopasowaniu schematu

$$\frac{\text{Fitch}(j) \quad \text{Fitch}(k) \quad \text{Fitch}(l)}{\text{Fitch}(i)} (\vee E)$$

do schematu reguły

$$\frac{\Gamma_1 \vdash \phi \vee \psi \quad \Gamma_2, \phi \vdash \rho \quad \Gamma_3, \psi \vdash \rho}{\Gamma_1, \Gamma_2, \Gamma_3 \vdash \rho} (\vee E)$$

poprawność zastosowania reguły $\forall E$ wymaga, aby teza sekwentu $\text{Fitch}(j)$ była alternatywą, żeby lewa strona tej alternatywy znajdowała się w kontekście sekwentu $\text{Fitch}(k)$, a prawa w kontekście sekwentu $\text{Fitch}(l)$, a także żeby tezy sekwentów $\text{Fitch}(k)$ oraz $\text{Fitch}(l)$ były takie same.

Podobne warunki wynikają dla wszystkich reguł wnioskowania i sprowadzają się do sprawdzenia, czy struktura użytych sekwentów odpowiada instancji danej reguły.

W związku z tym konieczne jest wprowadzenie formalizmu, który pozwala jednoznacznie odtworzyć sekwent odpowiadający każdej linii dowodu, sprawdzić poprawność ich dopasowania do reguł wnioskowania, a także zweryfikować, czy formuła zapisana w danej linii odpowiada tezie kodowanego przez nią sekwentu.

Rozdział 2.

Reprezentacja formuł, sekwentów, reguł wnioskowania i dowodów

Pierwszym etapem konstrukcji systemu było zdefiniowanie reprezentacji obiektów logicznych, na bazie których wykonywane będą dalsze operacje: parsowanie dowodów, sprawdzanie poprawności zastosowań reguł, generowanie danych treningowych oraz weryfikacja kroków proponowanych przez model językowy. Reprezentacja ta musiała spełniać dwa podstawowe wymagania. Po pierwsze, powinna wiernie odpowiadać formalizmowi dedukcji naturalnej opisanemu w poprzednim rozdziale. Po drugie, powinna umożliwiać łatwe przekształcanie obiektów logicznych na postać tekstową, ponieważ właśnie na takich tekstowych reprezentacjach uczony jest model językowy.

2.1. Reprezentacja formuł i sekwentów

Podstawowymi obiektami składniowymi są zmienne, relacje oraz formuły. W obecnej wersji systemu wykorzystywane są jedynie zmienne zdaniowe, jednak reprezentacja została zaprojektowana w sposób nieco ogólniejszy. Atom traktowany jest jako zastosowanie relacji do listy argumentów. W przypadku rachunku zdań wystarcza relacja jednoargumentowa, dlatego zapis taki jak x może być interpretowany jako uproszczona postać atomu. Takie rozwiązanie nie jest konieczne dla samego rachunku zdań, ale ułatwia ewentualne rozszerzenie systemu o bardziej ogólną składnię, na przykład o predykaty wieloargumentowe.

Formuły złożone reprezentowane są rekurencyjnie. Formuła atomowa jest przypadkiem bazowym, natomiast negacja, koniunkcja, alternatywa, implikacja i równoważność przechowują swoje bezpośrednie podformuły. Dzięki temu struktura danych odpowiada drzewiastej strukturze składniowej formuły. Taka reprezentacja pozwala

zarówno łatwo analizować postać formuły, jak i wykonywać operacje potrzebne przy stosowaniu reguł wnioskowania, na przykład odczytanie lewej i prawej strony implikacji czy koniunkcji.

Kontekst dowodzenia reprezentuje zbiór formuł przyjętych jako założenia danego sekwentu. W implementacji został on podzielony na dwie części: kontekst bazowy oraz kontekst dodatkowy. Kontekst bazowy zawiera formuły traktowane jako założenia obowiązujące w całym rozważanym fragmencie dowodu. Kontekst dodatkowy zawiera natomiast formuły wprowadzane lokalnie przez reguły wnioskowania, na przykład przy wprowadzaniu implikacji, negacji albo przy rozumowaniu nie wprost.

Podział ten nie zmienia znaczenia logicznego sekwentu, ale ma znaczenie techniczne i algorytmiczne. Pozwala odróżnić założenia, które powinny być stale dostępne podczas generowania poddowodu, od założeń lokalnych, które mogą być wprowadzane i usuwane przez konkretne reguły. Jest to szczególnie istotne później, gdy system będzie rozbił jeden problem dowodowy na podproblemy oraz generował dowody sekwentów z niepustym kontekstem.

Zdefiniowany wcześniej sekwent jest w systemie reprezentowany jako para złożona z kontekstu oraz tezy. W implementacji obiekt ten nosi nazwę `LittleProblem`, ponieważ każdy sekwent można traktować jako mały problem dowodowy: należy wyprowadzić określoną tezę przy założeniu określonego kontekstu. Taka nazwa jest szczególnie naturalna w dalszej części pracy, gdzie większe problemy dowodowe będą rozbijane na mniejsze podproblemy.

Reprezentacja sekwentu jest bezpośrednio wykorzystywana przez wszystkie dalsze komponenty programu: reguły wnioskowania przekształcają listy sekwentów-przesłanek w sekwenty-konkluzje, parser odtwarza sekwenty zakodowane w kolejnych liniach dowodu, a weryfikator sprawdza, czy dana linia rzeczywiście odpowiada sekwentowi otrzymanemu przez zastosowanie wskazanej reguły.

2.2. Reprezentacja reguł wnioskowania

Reguły wnioskowania zostały zaimplementowane jako osobne obiekty odpowiadające regułom systemu przedstawionym w Tabeli 1.1., a także pomocniczym regułom `FromContext` oraz `FromWeakenContext`. Na poziomie programu każda reguła działa przede wszystkim jako funkcja częściowa: dla poprawnej listy sekwentów-przesłanek zwraca sekwent będący konkluzją zastosowania tej reguły, natomiast dla danych, które nie pasują do jej schematu, zgłasza błąd.

Można to opisać następującym schematem:

Algorithm 1 Zastosowanie reguły wnioskowania

```

1: procedure ZASTOSUJREGULĘ( $R, P, A$ )
2:   if  $P$  nie spełnia warunków wejściowych reguły  $R$  then
3:     zgłoś błąd
4:   end if
5:    $S \leftarrow$  konkluzja reguły  $R$  dla przesłanek  $P$  i argumentów  $A$ 
6:   return  $S$ 
7: end procedure

```

Warunki wejściowe obejmują zgodność liczby przesłanek, zgodności ich typów (każda przesłanka musi być sekwentem), oraz ich zgodności ze schematem danej reguły. Argument R oznacza regułę wnioskowania, P oznacza listę sekwentów-przesłanek, natomiast A oznacza dodatkowe dane potrzebne w tych regułach, których zastosowanie nie jest jednoznacznie wyznaczone przez same przesłanki. Przykładowo w regule $(\forall I_1)$ z samej przesłanki można odczytać jedynie formułę φ , ponieważ przesłanka ma postać $\Gamma \vdash \varphi$. Aby skonstruować konkluzję

$$\Gamma \vdash \varphi \vee \psi,$$

system musi dodatkowo znać formułę ψ . Nie jest ona wyznaczona przez przesłankę, dlatego jest przekazywana jako dodatkowy argument zastosowania reguły. Analogiczna sytuacja występuje w regule $(\forall I_2)$, gdzie dodatkowym argumentem jest lewy składnik alternatywy.

Istotne jest to, że niepoprawne zastosowanie reguły nie jest reprezentowane przez wartość logiczną **False**. Zamiast tego reguła zgłasza błąd. Jest to wygodne z punktu widzenia dalszej konstrukcji systemu, ponieważ zastosowanie reguły można traktować jako operację, która albo zwraca poprawnie skonstruowany sekwent, albo w ogóle nie jest zdefiniowana dla podanych argumentów. Dzięki temu w pozostałych częściach programu nie trzeba po każdym zastosowaniu reguły osobno sprawdzać, czy otrzymany wynik jest poprawny. Błędne kandydatury mogą być po prostu odrzucane przez przechwycenie błędu.

2.3. Reprezentacja dowodów

W poprzednim rozdziale przyjęliśmy, że każda linia dowodu koduje jeden sekwent. Na pierwszy rzut oka mogłoby się więc wydawać, że po sparsowaniu dowodu wystarczy przechowywać wyłącznie sekwenty odpowiadające kolejnym liniom. Takie rozwiązanie pozwalałoby odtworzyć, jakie sekwenty zostały kolejno otrzymane, ale traciłoby informację o tym, jak poszczególne kroki zostały uzasadnione.

Sama reprezentacja sekwentu jest więc zbyt uboga jako reprezentacja linii dowodu. Sekwent jest logicznym rezultatem danej linii, natomiast linia zawiera również

dane potrzebne do odtworzenia tego rezultatu: numer linii, formułę zapisaną jako jej tezę, użytą regułę wnioskowania oraz indeksy występujące w uzasadnieniu. Bez tych informacji sparsowany dowód byłby jedynie ciągiem otrzymanych sekwentów, a nie strukturą pozwalającą odtworzyć zależności między krokami.

Sparsowana linia dowodu nie przechowuje oryginalnego tekstu linii, lecz informacje wydobyte z tego tekstu oraz sekwent odtworzony na ich podstawie. Do informacji odczytanych z zapisu należą: numer linii, lista indeksów występujących w jej uzasadnieniu, obiekt reprezentujący zastosowaną regułę oraz formuła zapisana jako teza linii. Na podstawie tych danych, a także wcześniej sparsowanej części dowodu, konstruowany jest sekwent kodowany przez daną linię.

Można to przedstawić schematycznie w następujący sposób:

$$\text{zapis linii} \longrightarrow (i, \varphi, R, J) \longrightarrow S_i.$$

Tutaj i oznacza numer aktualnej linii, φ formułę zapisaną w tej linii, R zastosowaną regułę, natomiast $J = (j_1, \dots, j_n)$ oznacza ciąg indeksów podanych w uzasadnieniu linii. Indeksy te nie muszą być dokładnie listą przesłanek reguły wnioskowania, ale w zależności od reguły mogą również zawierać indeksy linii z których czerpiemy dodatkowe argumenty do wywołania tej reguły, które nie są przesłankami. Na przykład linia postaci

$$\text{i. } \phi \quad \text{weakening, j, k}$$

zawiera dwa indeksy, ale tylko jeden z nich wskazuje właściwą przesłankę reguły. Sekwent S_k jest sekwentem, który ma zostać osłabiony, natomiast linia j służy do odczytania formuły dodawanej do kontekstu. Jeśli więc $S_k = \Gamma \vdash \phi$, a tezą linii j jest ψ , to system konstruuje sekwent

$$\Gamma, \psi \vdash \phi.$$

W tym przypadku dodatkowa formuła ψ nie pochodzi z formuły zapisanej w aktualnej linii, lecz z tezy wcześniejszej linii wskazanej przez jeden z indeksów w J .

Innym przykładem jest linia postaci

$$\text{i. } \phi \vee \psi \quad \vee\text{-introduction1, j}$$

Tutaj indeks j wskazuje właściwą przesłankę, czyli wcześniejszy sekwent $S_j = \Gamma \vdash \varphi$. Sama przesłanka nie wyznacza jednak formuły ψ , która ma zostać dodana jako prawa strona alternatywy. System odczytuje ją więc z formuły zapisanej w aktualnej linii, czyli z prawego składnika $\varphi \vee \psi$, i przekazuje ją jako dodatkowy argument reguły.

Jeszcze innym przypadkiem są reguły, które nie odwołują się do żadnej wcześniejszej linii. Przykładem jest linia postaci

$$1. \phi \text{ assumption}$$

W najprostszym przypadku odpowiada ona sekwentowi $\phi \vdash \phi$. W naszym systemie sekwent konstruowany dla takiej linii może jednak zawierać również niepusty kontekst bazowy, obowiązujący w całym aktualnie rozważanym fragmencie dowodu. Tego kontekstu nie da się odczytać z wcześniejszych sparsowanych linii, ponieważ takowe jeszcze nie istnieją. Dlatego kontekst bazowy musi zostać przekazany *explicite* jako dodatkowy argument podczas konstruowania linii.

Sparsowany dowód jest przechowywany jako słownik, który mapuje indeksy linii na odpowiadające im sparsowane linie. Jeżeli przez D oznaczymy taki słownik, to $D[i]$ jest obiektem reprezentującym linię o numerze i , a $D[i].LP$ oznacza sekwent odtworzony dla tej linii. Dzięki temu podczas parsowania kolejnej linii wystarczy odwołać się do indeksów zapisanych w jej uzasadnieniu i pobrać z D potrzebne wcześniej skonstruowane obiekty.

Schematycznie parsowanie dowodu można więc opisać następująco:

$$D_0 = \emptyset,$$

$$D_i = D_{i-1} \cup \{i \mapsto L_i\},$$

gdzie L_i jest sparsowaną postacią i -tej linii dowodu, skonstruowaną przy użyciu słownika D_{i-1} . Oznacza to, że każda kolejna linia może korzystać wyłącznie z informacji odtworzonych dla wcześniejszych linii. Taki sposób parsowania odpowiada liniowej strukturze dowodu: późniejsze kroki mogą odwoływać się do wcześniejszych, ale nie odwrotnie.

Po skonstruowaniu sparsowanej linii system sprawdza jeszcze, czy teza odtworzonego sekwentu jest identyczna z formułą zapisaną w tekście linii. Ten krok jest konieczny, ponieważ formuła występująca w zapisie linii jest traktowana jako deklaracja, a nie jako samodzielne źródło informacji o całym sekwencie. Jeżeli reguła zastosowana do odtworzonych przesłanek i argumentów prowadzi do innej tezy niż ta zapisana w linii, linia zostaje odrzucona. W ten sposób parser pełni jednocześnie rolę weryfikatora poprawności lokalnych kroków dowodu.

Niepoprawna linia nie jest oznaczana specjalną wartością logiczną, lecz powoduje zgłoszenie błędu. Dotyczy to zarówno błędów składniowych, jak i sytuacji, w których indeksy zapisane w uzasadnieniu nie pozwalają skonstruować poprawnego zastosowania wskazanej reguły. Takie rozwiązanie upraszcza późniejsze etapy działania systemu. Podczas generowania korpusu lub sprawdzania linii proponowanych przez model językowy można po prostu próbować skonstruować sparsowaną linię; jeśli konstrukcja się powiedzie, linia jest poprawna, a jeśli pojawi się błąd, kandydata zostaje odrzucona.

Ta sama reprezentacja jest wykorzystywana przy generowaniu korpusu. Program nie tworzy linii treningowych przez ręczne sklejanie napisów, lecz najpierw konstruuje poprawny obiekt linii, a dopiero potem wypisuje go w ustalonym formacie tekstowym. Dzięki temu każda wygenerowana linia ma od razu odpowiadającą jej strukturę logiczną, którą parser potrafi ponownie odtworzyć.

2.3.1. Modele językowe

W niniejszej pracy jednym z podstawowych narzędzi wykorzystywanych do konstrukcji systemu dowodzącego są modele językowe. Modelem językowym nazywamy model, którego zadaniem jest opisywanie regularności występujących w tekstach. W praktyce można go rozumieć jako system, który dla danego początkowego fragmentu tekstu przewiduje, jakie elementy mogą pojawić się dalej. W typowych zastosowaniach tekstem takim może być tekst w języku naturalnym, fragment programu komputerowego albo odpowiedź na pytanie. W tej pracy tekstem generowanym przez model będą natomiast dowody w ustalonej notacji oraz zapisy rozbić problemów logicznych na prostsze podproblemy.

Tekst przekazywany do modelu nie jest zwykle analizowany jako ciąg znaków w najprostszym sensie. Najpierw zostaje on podzielony na tokeny, czyli jednostki, na których bezpośrednio operuje model. Tokenami mogą być pojedyncze znaki, fragmenty słów, całe słowa albo inne fragmenty tekstu zależnie od przyjętej metody kodowania. Po takim zakodowaniu tekst można traktować jako ciąg tokenów. Zadaniem modelu językowego jest wtedy przewidywanie kolejnych tokenów na podstawie tokenów wcześniejszych.

W pracy korzystam z modeli autoregresyjnych. Model autoregresyjny generuje tekst krok po kroku. Mając pewien ciąg tokenów wejściowych, wyznacza rozkład prawdopodobieństwa następnego tokenu. Następnie wybrany token jest dopisywany do dotychczasowego tekstu i cały proces może zostać powtórzony. W ten sposób model, który w pojedynczym kroku przewiduje tylko jeden następny token, może wygenerować dłuższy fragment tekstu. Modele typu GPT są właśnie przykładami takich modeli autoregresyjnych opartych na architekturze transformerów [?, ?].

Trenowanie modelu językowego polega na uczeniu go przewidywania kolejnych tokenów w tekstach pochodzących z korpusu treningowego. Model otrzymuje fragment tekstu i ma przewidzieć tokeny występujące po nim. Następnie jego parametry są zmieniane tak, aby zwiększać prawdopodobieństwo tych tokenów, które rzeczywiście wystąpiły w danych treningowych. Można więc powiedzieć, że model uczy się regularności obecnych w korpusie: zarówno prostych zależności lokalnych między sąsiednimi tokenami, jak i bardziej złożonych wzorców pojawiających się w dłuższych fragmentach tekstu.

W niniejszej pracy szczególnie istotne jest to, że tekstem, którego regularności ma nauczyć się model, nie jest zwykły tekst w języku naturalnym, lecz zapis dowodu albo zapis rozbięcia problemu logicznego. Dowody w rozważanej notacji mają bardzo regularną strukturę: każda linia zawiera formułę, nazwę zastosowanej reguły oraz indeksy wcześniejszych linii, do których dana reguła się odwołuje. Podobnie zapisy rozbić problemów na podproblemy mają ustalony format tekstowy. Pozwala to potraktować generowanie kolejnych kroków dowodu jako szczególny przypadek generowania tekstu.

Jako bazę implementacyjną wykorzystuję projekt **nanoGPT** [?]. Jest to niewielka implementacja modelu typu GPT, która pozwala trenować modele językowe na własnych korpusach tekstowych. W moim systemie trenowane są modele wyspecjalizowane do konkretnych zadań związanych z dowodzeniem. Jeden z nich służy do generowania dowodów sekwentów z założeniami, natomiast drugi do generowania rozbić problemów na prostsze podproblemy. Oba modele uczone są na specjalnie przygotowanych korpusach, których konstrukcja zostanie omówiona w dalszej części pracy.

Należy jednak podkreślić, że model językowy nie jest w tej pracy traktowany jako samodzielny system dowodzący. Sam fakt, że model wygenerował tekst przypominający dowód, nie oznacza jeszcze, że jest to dowód poprawny formalnie. Model może wskazać niewłaściwą regułę, odwołać się do niepasujących linii albo wygenerować formułę, która nie jest konkluzją wynikającą z podanych przesłanek. Dlatego model pełni rolę komponentu heurystycznego: proponuje kolejne kroki dowodu albo możliwe rozbięcia problemu, natomiast ich poprawność jest sprawdzana przez część symboliczną systemu.

Takie połączenie modelu językowego i formalnego weryfikatora jest naturalne dla rozważanego problemu. Z jednej strony klasyczne, czysto symboliczne przeszukiwanie przestrzeni dowodów może być trudne, ponieważ często nie wiadomo, którą regułę wnioskowania zastosować i jakie formuły pomocnicze należy wprowadzić. Z drugiej strony model językowy może nauczyć się z korpusu, jakie kroki dowodowe często pojawiają się w podobnych sytuacjach. Nie gwarantuje on jednak poprawności tych kroków. Weryfikator pełni więc rolę filtra, który odrzuca propozycje niepoprawne i pozwala wykorzystać tylko te fragmenty wygenerowanego tekstu, które rzeczywiście kodują poprawne zastosowania reguł wnioskowania.

W rezultacie system nie polega wyłącznie na generowaniu tekstu przez model, ale łączy dwa podejścia. Część neuronowa służy do podpowiadania potencjalnych kierunków dowodu, natomiast część symboliczna odpowiada za jednoznaczną interpretację wygenerowanych linii i sprawdzenie ich poprawności. Dzięki temu można wykorzystać zdolność modeli językowych do rozpoznawania regularności w danych tekstowych, jednocześnie zachowując formalną kontrolę nad poprawnością konstruowanych dowodów.

Rozdział 3.

Strategie konstruowania dowodów

3.1. Motywacja

Celem systemu nie jest jedynie wygenerowanie tekstu przypominającego dowód w dedukcji naturalnej, lecz znalezienie poprawnego dowodu danej tautologii. Model językowy pełni w tym procesie rolę generatora kandydatów: proponuje kolejne linie dowodu albo inne kroki pomocnicze, ale nie decyduje samodzielnie o ich poprawności. Każdy wygenerowany fragment musi zostać zweryfikowany przez opisany wcześniej parser i mechanizm sprawdzania reguł wnioskowania.

Takie rozdzielenie ról jest kluczowe. Model językowy może dobrze naśladować składnię dowodów i często proponować sensowne kroki, ale sam z siebie nie gwarantuje poprawności logicznej. Może wygenerować linię o poprawnym wyglądzie, która jednak odwołuje się do niewłaściwych wcześniejszych linii, używa reguły w niepoprawny sposób albo wprowadza formułę, której nie da się uzasadnić w danym kontekście. Dlatego w systemie model nie jest traktowany jako samodzielny system, lecz jako źródło propozycji filtrowanych przez formalny weryfikator.

Najprostsze podejście polega na liniowym dopisywaniu kolejnych linii dowodu. System przekazuje modelowi cel dowodowy oraz dotychczas wygenerowany fragment dowodu, a następnie próbuje dopisać jedną nową linię. Jeżeli po jej dopisaniu dowód nadal poprawnie się parsuje, linia zostaje zaakceptowana; w przeciwnym razie jest odrzucana. Proces generacji trwa do momentu uzyskania linii kodującej poszukiwany sekwent albo do chwili, w której dalsze rozszerzanie dowodu zostaje uznane za niecelowe. Tak powstaje pierwsza strategia, w której dowód konstruowany jest jako jedna rosnąca sekwencja lokalnie poprawnych kroków.

Podejście to ma jednak istotne ograniczenie. Konstruowanie dowodu rzadko jest procesem całkowicie liniowym. Nawet jeśli każda kolejna linia jest poprawna, może

okazać się, że wybrana ścieżka nie prowadzi do celu albo prowadzi do niego bardzo okreżną drogą. W praktyce potrzebne jest więc pewne przeszukiwanie przestrzeni możliwych częściowych dowodów: system powinien móc rozwijać kilka alternatywnych prób i wracać do bardziej obiecujących fragmentów. Tę ideę realizuje druga strategia, która przechowuje wiele częściowych dowodów i wybiera spośród nich te, które warto dalej rozwijać.

Dla trudniejszych formuł nawet takie lokalne przeszukiwanie może być niewystarczające. Problemem nie jest wtedy tylko wybór następnej linii, lecz wybór ogólnego planu dowodu. Często łatwiej jest najpierw zdecydować, jaką regułą można byłoby otrzymać cel, a następnie spróbować udowodnić jej przesłanki jako osobne podproblemy. W ten sposób otrzymujemy trzecią strategię, w której system może rozbić początkowy problem na prostsze sekweny, a następnie szukać dowodów dla nich. Strategia ta łączy generowanie dowodów w notacji liniowej z przeszukiwaniem drzewa problemów i podproblemów.

Kolejne strategie powstały więc jako odpowiedź na ograniczenia poprzednich. Pierwsza strategia sprawdza, czy model potrafi lokalnie dopisywać poprawne linie dowodu. Druga dodaje mechanizm wyboru między wieloma częściowymi próbami, ale nadal wykorzystuje pierwszą strategię jako procedurę rozwijania pojedynczego fragmentu dowodu. Trzecia zmienia poziom poszukiwania: oprócz dopisywania linii pozwala planować dowód przez dekompozycję celu na podproblemy. Również ona nie zastępuje wcześniejszych metod, lecz korzysta z nich jako komponentów pomocniczych: po rozbiciu celu na podproblemy próbujemy znaleźć dowody tych podproblemów przy użyciu drugiej strategii.

W ten sposób strategie tworzą hierarchię. Każda kolejna strategia rozszerza poprzednią, traktując ją jako niezależny komponent realizujący prostsze zadanie. Dzięki temu system można opisywać warstwowo: najpierw jako generator pojedynczej poprawnej kontynuacji dowodu, następnie jako mechanizm przeszukiwania wielu takich kontynuacji, a ostatecznie jako procedurę rozbijania problemu dowodowego na mniejsze problemy, dla których ponownie uruchamiane są wcześniejsze mechanizmy. W dalszej części rozdziału strategie te zostaną opisane w tej kolejności.

3.2. Strategia pierwsza: liniowe dopisywanie kroków dowodu

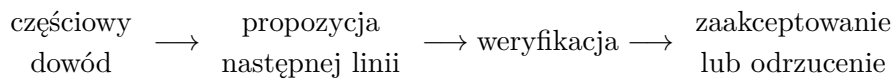
3.2.1. Idea strategii

Pierwsza strategia jest najprostszym sposobem wykorzystania modelu językowego w systemie dowodzącym. Jej celem jest sprawdzenie, czy model potrafi generować kolejne poprawne kroki dowodu w notacji liniowej, jeżeli otrzyma opis celu oraz dotychczasowy fragment dowodu. Strategia ta nie próbuje planować całego do-

wodu z góry ani nie rozważa wielu alternatywnych ścieżek. Dowód jest konstruowany sekwencyjnie: w każdym kroku system próbuje dopisać jedną nową linię do końca aktualnego fragmentu.

W tym sensie strategia pierwsza traktuje model jako lokalny generator kontynuacji. Wejściem dla modelu jest prompt kodujący sekwent, który ma zostać udowodniony, oraz ewentualnie dotychczas wygenerowane linie dowodu. Wyjściem nie jest cały dowód, lecz kandydatura na następną linię. Poprawność tej kandydatury nie jest oceniana przez model, lecz przez formalny parser i mechanizm sprawdzania reguł wnioskowania.

Schematycznie można to przedstawić następująco:



W implementacji tej strategii odpowiada podstawowa procedura lokalnego rozwijania częściowego dowodu, oznaczona jako `silliest_generate_proof`. Nazwa ta odzwierciedla prototypowy charakter pierwszej wersji algorytmu: strategia jest celowo prosta i służy jako punkt wyjścia dla bardziej złożonych metod opisanych w dalszej części rozdziału.

3.2.2. Generowanie i weryfikacja pojedynczej linii

Podstawowa operacja wykonywana przez strategię polega na próbie dopisania jednej nowej linii do aktualnego fragmentu dowodu. Nie odbywa się to jednak przez zupełnie swobodne wygenerowanie dowolnego ciągu znaków przez model. System narzuca częściową strukturę generowanej linii, tak aby model odpowiadał przede wszystkim za wybór formuły i uzasadnienia, a nie za odtwarzanie elementów czysto technicznych, takich jak numeracja linii.

Najpierw analizowany jest aktualny prompt. Jeżeli zawiera on wyłącznie cel dowodowy, następna linia otrzymuje numer 1. Jeżeli w prompcie znajdują się już wcześniejsze linie dowodu, system odczytuje numer ostatniej z nich i rozpoczyna nową linię od kolejnego numeru. Dzięki temu model nie musi samodzielnie utrzymywać poprawnej numeracji dowodu, co zmniejsza liczbę błędów czysto składniowych.

Następnie generowana jest formuła zapisana w nowej linii. Na tym etapie system ogranicza zbiór akceptowanych tokenów. Dopuszczalne są symbole logiczne, nawiasy, stałe $\top$ i $\perp$, spacje oraz zmienne występujące w dowodzonym sekwencie. Jeżeli dowód ma niepusty kontekst bazowy, zbiór dopuszczalnych zmiennych jest wyznaczany na podstawie zarówno tezy, jak i formuł występujących w kontekście. Celem tego ograniczenia jest uniknięcie sytuacji, w której model wprowadza do dowodu nowe, przypadkowe zmienne, niezwiązane z rozważanym problemem.

Generowanie formuły trwa do momentu, w którym pojawi się separator oddzielający formułę od nazwy reguły uzasadniającej linię. W przyjętym formacie rolę tę

pełnią cztery kolejne spacje. Po ich wygenerowaniu system przechodzi do drugiej części linii, czyli do uzasadnienia zawierającego nazwę reguły oraz ewentualne indeksy wcześniejszych linii. Ta część jest generowana już jako kontynuacja rozpoczętej linii i kończy się w momencie wygenerowania znaku nowej linii.

Można to opisać schematycznie następująco:

prompt $\longrightarrow$ numer nowej linii $\longrightarrow$ formuła $\longrightarrow$ uzasadnienie.

W implementacji ten etap odpowiada procedurze generowania następnej linii, która na podstawie aktualnego prompta tworzy kandydaturę na kolejną linię dowodu.

Po wygenerowaniu kandydackiej linii system dopisuje ją tymczasowo do aktualnego fragmentu dowodu i próbuje ponownie sparsować całość. Sprawdzenie to nie ogranicza się do kontroli składni. Parser próbuje odtworzyć strukturę każdej linii, skonstruować odpowiadające jej sekweny i zweryfikować, czy zastosowane reguły rzeczywiście prowadzą do zapisanych tez. Jeżeli parsowanie kończy się błędem, wygenerowana linia zostaje odrzucona.

Kandydacka linia może zostać odrzucona z kilku powodów. Po pierwsze, może nie spełniać wymagań składniowych przyjętego formatu. Po drugie, może zawierać formułę niezgodną z ograniczeniami na zmienne występujące w dowodzonym problemie (sprawdzenie tego warunku jest defacto nadmiarowe, ponieważ jest on sprawdzany już na etapie generowania linii). Po trzecie, może wyglądać jak poprawna linia dowodu, ale odwoływać się do wcześniejszych linii w sposób niezgodny ze schematem wskazanej reguły. W każdym z tych przypadków parser lub weryfikator zgłasza błąd, a wygenerowana kandydatura nie jest dopisywana do dowodu.

Jeżeli natomiast dopisanie linii nie powoduje błędu parsowania, linia zostaje zaakceptowana. Oznacza to, że udało się odtworzyć sekwent kodowany przez tę linię, a jego teza zgadza się z formułą zapisaną w tekście. Po zaakceptowaniu linii system traktuje ją tak samo jak wcześniejsze linie: może ona zostać wykorzystana jako uzasadnienie kolejnych kroków dowodu.

W praktyce strategia działa więc jako pętla generowania i filtrowania. Jeśli przez D_k oznaczmy aktualny częściowy dowód, a przez ℓ kandydaturę na nową linię, to po jej tymczasowym dopisaniu otrzymujemy

$$D_{k+1}^* = D_k + \ell.$$

Następnie system wykonuje weryfikację:

$$D_{k+1} = \begin{cases} D_{k+1}^*, & \text{jeśli } D_{k+1}^* \text{ jest poprawny,} \\ D_k, & \text{w przeciwnym przypadku.} \end{cases}$$

Takie rozwiązanie pozwala oddzielić zadanie generowania od zadania sprawdzania. Model językowy odpowiada za proponowanie potencjalnie sensownych kontynuacji, natomiast poprawność logiczna pozostaje kontrolowana przez formalny mechanizm opisany w poprzednim rozdziale.

3.2.3. Warunki zakończenia

Strategia kończy działanie sukcesem wtedy, gdy ostatnia zaakceptowana linia koduje sekwent docelowy. Innymi słowy, po sparsowaniu aktualnego fragmentu dowodu system sprawdza, czy sekwent odtworzony dla ostatniej linii jest równy sekwentowi, który miał zostać udowodniony. Jeśli tak jest, wygenerowany tekst stanowi pełny dowód celu.

Działanie strategii może zakończyć się także bez znalezienia dowodu. Pierwszym ograniczeniem jest maksymalna liczba nowych linii, które wolno dopisać do prompta. Chroni to system przed generowaniem bardzo długich fragmentów, które lokalnie pozostają poprawne, ale nie zbliżają się do celu. Drugim ograniczeniem jest maksymalna liczba kolejnych nieudanych prób dopisania linii. Jeżeli model przez wiele prób z rzędu generuje kandydatury odrzucane przez parser, system uznaje, że dalsze kontynuowanie tej samej ścieżki jest mało obiecujące.

W przypadku zakończenia bez sukcesu strategia może mimo to zwrócić częściowy dowód, czyli prompt z dopisanymi poprawnymi liniami. Taki fragment nie musi jeszcze dowodzić celu, ale może stanowić postęp względem punktu wyjścia. Właśnie dlatego pierwsza strategia może być później użyta jako komponent w strategii drugiej: nie musi samodzielnie zawsze znaleźć pełnego dowodu, wystarczy, że potrafi lokalnie rozwijać częściowy dowód o kolejne poprawne kroki.

3.2.4. Ograniczenia strategii

Największym ograniczeniem pierwszej strategii jest jej liniowy charakter. System rozwija tylko jedną ścieżkę dowodu i nie przechowuje alternatywnych kontynuacji. Jeżeli model wybierze poprawną, ale mało użyteczną linię, strategia nie ma mechanizmu powrotu do wcześniejszego stanu i wyboru innej możliwości. Może więc utknać w ciągu kroków, które są lokalnie poprawne, ale nie prowadzą do dowodu celu.

Drugim ograniczeniem jest brak globalnego planowania. Strategia nie analizuje struktury celu w taki sposób, aby zdecydować, jakie podcele warto najpierw osiągnąć. Każdy krok jest generowany na podstawie dotychczasowego tekstu dowodu, ale system nie rozważa jawnie, jaką regułą można byłoby zakończyć dowód ani jakie przesłanki należałoby wcześniej udowodnić. W prostych przypadkach lokalne dopisywanie linii może wystarczyć, ale dla bardziej złożonych formuł często potrzebne jest planowanie na wyższym poziomie.

Skuteczność tej strategii zależy również od parametrów generacji, takich jak temperatura oraz liczba dopuszczalnych prób wygenerowania kolejnej linii. Nie jest to jednak osobny problem konstrukcyjny, lecz ogólna konsekwencja użycia modelu językowego jako generatora kandydatów.

Mimo tych ograniczeń strategia pierwsza jest ważnym elementem całego systemu. Stanowi podstawowy mechanizm lokalnego rozszerzania dowodu, który może być używany jako komponent w bardziej złożonych metodach. Druga strategia zachowuje ten mechanizm generowania pojedynczych kroków, ale dodaje możliwość rozwijania wielu częściowych dowodów równolegle, co częściowo rozwiązuje problem utknięcia w jednej niekorzystnej ścieżce.

3.3. Strategia druga: przeszukiwanie wielu częściowych dowodów

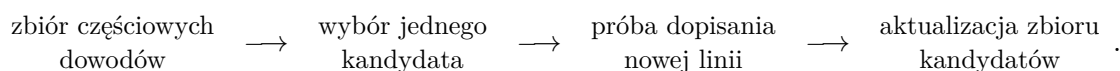
3.3.1. Motywacja

Pierwsza strategia konstruuje tylko jeden częściowy dowód. Oznacza to, że każda zaakceptowana linia staje się trwałą częścią aktualnej próby. Jeżeli później okaże się, że dana kontynuacja nie prowadzi w stronę rozwiązania, system nie ma możliwości powrotu do wcześniejszego etapu i wyboru innej ścieżki rozumowania. W praktyce jest to poważne ograniczenie, ponieważ poprawność lokalna pojedynczej linii nie gwarantuje jeszcze, że linia ta będzie użyteczna dla całego dowodu.

Druga strategia próbuje ograniczyć ten problem przez jednoczesne przechowywanie wielu częściowych dowodów. Zamiast rozwijać wyłącznie jedną sekwencję linii, system utrzymuje zbiór kandydatów. W każdej iteracji wybierany jest jeden częściowy dowód, a następnie podejmowana jest próba dopisania do niego kolejnej poprawnej linii. Jeżeli powstały w ten sposób fragment jest poprawny i nie wystąpił wcześniej, zostaje dodany do zbioru kandydatów.

Można więc powiedzieć, że druga strategia zmienia charakter generowania dowodu z liniowego dopisywania kolejnych kroków na przeszukiwanie przestrzeni częściowych dowodów. Pojedynczy nieoptymalny wybór nie przekreśla całego procesu, ponieważ inne częściowe dowody pozostają dostępne i mogą zostać rozwinięte w późniejszych iteracjach.

Schematycznie można to przedstawić następująco:



3.3.2. Zbiór częściowych dowodów

Podstawową strukturą wykorzystywaną przez drugą strategię jest zbiór częściowych dowodów. Na początku zawiera on jedynie prompt opisujący sekwent, który należy udowodnić. W kolejnych iteracjach do zbioru dodawane są nowe częściowe dowody powstałe przez dopisanie poprawnej linii do jednego z wcześniej istniejących kandydatów.

Każdy element tego zbioru jest tekstem zawierającym opis celu dowodowego oraz dotychczas wygenerowany fragment dowodu. Dzięki temu może zostać bezpośrednio podany modelowi językowemu jako kontekst generacji.

W każdej iteracji system wykonuje następujące kroki:

Algorithm 2 Rozwijanie zbioru częściowych dowodów

- 1: Wybierz częściowy dowód D ze zbioru kandydatów P .
 - 2: Spróbuj dopisać do D jedną nową poprawną linię, otrzymując D' .
 - 3: Sprawdź poprawność otrzymanego fragmentu D' .
 - 4: **if** D' kończy dowód celu **then**
 - 5: **return** D'
 - 6: **else if** D' jest poprawnym nowym częściowym dowodem **then**
 - 7: Dodaj D' do zbioru kandydatów P .
 - 8: **end if**
-

Ważne jest, że druga strategia nie zastępuje mechanizmu generowania pojedynczej linii, lecz używa go jako podprocedury. Próba rozwinięcia częściowego dowodu nadal polega na tym, że model proponuje kandydacką linię, a formalny parser i weryfikator sprawdzają jej poprawność. Różnica polega na tym, że poprawne rozwinięcia nie muszą tworzyć jednej liniowej historii. Mogą współistnieć jako alternatywne ścieżki poszukiwania dowodu.

3.3.3. Ocena prawdopodobieństwa częściowego dowodu

Jeżeli zbiór częściowych dowodów zawiera wiele kandydatów, trzeba zdecydować, który z nich rozwijać w następnej iteracji. Najprostsze rozwiązanie polegałoby na losowaniu jednostajnym, jednak ignorowałoby ono to, że nie wszystkie częściowe dowody są tak samo prawdopodobne. Model nie tylko generuje tekst, ale również przypisuje kolejnym tokenom prawdopodobieństwa. Można więc użyć tych prawdopodobieństw jako przybliżonej oceny tego, jak naturalny dla modelu jest dany częściowy dowód.

Dla każdego częściowego dowodu obliczana jest wartość zależna od prawdopodobieństw tokenów tworzących wygenerowaną część dowodu. Ponieważ iloczyn wielu prawdopodobieństw bardzo szybko staje się liczbą bliską zero, wygodniej sumować logarytmy prawdopodobieństw. Aby nie faworyzować nadmiernie krótkich fragmentów tylko dlatego, że składają się z mniejszej liczby tokenów, suma ta jest normalizowana przez długość analizowanego tekstu.

Jeżeli częściowy dowód oznaczmy przez D , a jego kolejne tokeny przez $t_1, \dots, t_m$, to jego ocenę można zapisać schematycznie jako

$$\text{score}(D) = \frac{1}{m} \sum_{i=1}^m \log P(t_i \mid t_1, \dots, t_{i-1}, \text{prompt celu}).$$

Wartość $score(D)$ jest jednak nieokreślona, gdy $m = 0$, czyli dla początkowego prompta, który nie zawiera jeszcze żadnych wygenerowanych tokenów. W takim przypadku najpierw wyliczamy $score$ dla pozostałych częściowych dowodów, a następnie ustawiamy $score$ dla pustego prompta na wartość średnią z tych ocen. Intuicja jest taka, że początkowy prompt nie powinien być ani stale faworyzowany, ani całkowicie pomijany. Z jednej strony nie zawiera on jeszcze żadnego postępu w konstrukcji dowodu, więc zbyt częste wybieranie go prowadziłoby do wielokrotnego rozpoczynania generacji od początku. Z drugiej strony jest to kandydat najbardziej neutralny: nie zawiera żadnych wcześniejszych decyzji, które mogłyby skierować dowód w mało użyteczną stronę. Przypisanie mu średniej oceny pozwala więc zachować możliwość powrotu do początku rozumowania, ale nie czyni z niego domyślnie najatrakcyjniejszej ścieżki.

Kolejnym problemem jest to, że wartości $score(D)$, jako średnie logarytmów prawdopodobieństw, są w praktyce niedodatnie. Mogą być równe 0 jedynie w granicznym przypadku, gdy każdy kolejny token ma prawdopodobieństwo warunkowe równe 1. Ponadto wartości te nie sumują się do 1, a więc nie mogą być bezpośrednio interpretowane jako rozkład prawdopodobieństwa. Aby przekształcić je w rozkład na zbiorze kandydatów, używamy funkcji softmax:

$$P(D_i) = \frac{\exp(score(D_i))}{\sum_j \exp(score(D_j))}.$$

W ten sposób każdemu częściowemu dowodowi przypisana zostaje dodatnia wartość, a suma wszystkich takich wartości wynosi 1. Kandydaci o wyższym wyniku $score$, czyli bardziej prawdopodobni według modelu, otrzymują większe prawdopodobieństwo wyboru w następnej iteracji.

3.3.4. Pełny przebieg strategii

Po wprowadzeniu zbioru częściowych dowodów oraz sposobu przypisywania im prawdopodobieństw wyboru można przedstawić całą drugą strategię jako jedną pętlę przeszukiwania. W każdej iteracji system aktualizuje oceny kandydatów, wybiera jeden z częściowych dowodów zgodnie z rozkładem otrzymanym po zastosowaniu funkcji softmax, a następnie próbuje rozwinąć go o jedną poprawną linię.

Algorithm 3 Przeszukiwanie wielu częściowych dowodów

```

1:  $P \leftarrow \{\text{prompt opisujący cel } G\}$ 
2: while nie przekroczono limitu iteracji do
3:   Oblicz wartości  $\text{score}(D)$  dla kandydatów  $D \in P$ .
4:   Przekształć wartości  $\text{score}(D)$  w rozkład prawdopodobieństwa na  $P$ .
5:   Wylosuj częściowy dowód  $D \in P$  zgodnie z otrzymanym rozkładem.
6:   Spróbuj dopisać do  $D$  jedną poprawną linię, otrzymując  $D'$ .
7:   if  $D'$  jest pełnym dowodem celu  $G$  then
8:     return  $D'$ 
9:   else if  $D'$  jest poprawnym nowym częściowym dowodem then
10:     $P \leftarrow P \cup \{D'\}$ 
11:   end if
12: end while
13: return informacja o niepowodzeniu

```

W powyższym schemacie etap dopisania nowej linii korzysta z procedury opisanej przy pierwszej strategii. Druga strategia nie zmienia więc sposobu weryfikacji pojedynczego kroku dowodu, lecz zmienia organizację całego procesu poszukiwania: zamiast jednej rozwijanej sekwencji utrzymywany jest zbiór alternatywnych prób, spośród których kolejne są wybierane probabilistycznie.

Niepowodzenie algorytmu oznacza jedynie, że dowód nie został znaleziony w ramach przyjętych ograniczeń obliczeniowych. Nie jest to informacja o niedowodliwości danego sekwentu.

3.3.5. Ograniczenia strategii

Druga strategia usuwa najważniejsze ograniczenie strategii liniowej: system nie jest już przywiązany do jednej ścieżki generacji. Może utrzymywać wiele alternatywnych częściowych dowodów i stopniowo rozwijać te, które wydają się bardziej obiecujące. Nie rozwiązuje to jednak wszystkich problemów związanych z automatycznym konstruowaniem dowodów.

Po pierwsze, strategia nadal ma charakter lokalny. Rozwijanie częściowego dowodu polega na dopisaniu kolejnej linii, a nie na świadomym zaplanowaniu całej struktury dowodu. System może więc generować poprawne, ale mało użyteczne kroki, które zwiększają długość dowodu bez realnego przybliżania go do celu.

Po drugie, ocena prawdopodobieństwa częściowego dowodu nie jest tym samym co ocena jego matematycznej przydatności. Model może wysoko oceniać fragmenty podobne do danych treningowych, ale nie oznacza to jeszcze, że są one najlepszą drogą do udowodnienia konkretnej tezy. Prawdopodobieństwo generacji jest więc heurystyką, a nie kryterium poprawności ani gwarancją sukcesu.

Po trzecie, liczba możliwych częściowych dowodów rośnie bardzo szybko. Każda zaakceptowana linia może prowadzić do kolejnych rozwinięć, a wiele z nich może być formalnie poprawnych. W rezultacie strategia wymaga limitów liczby iteracji oraz mechanizmów wyboru kandydatów do rozwijania. Limity te są konieczne obliczeniowo, ale sprawiają, że przeszukiwanie pozostaje niepełne.

Najważniejsze ograniczenie polega jednak na tym, że druga strategia nadal nie wykonuje dekompozycji celu na podcele. System próbuje znaleźć dowód przez rozszerzanie częściowych tekstowych dowodów, ale nie ma osobnego etapu, w którym analizuje strukturę dowodzonego sekwentu i wybiera regułę pozwalającą rozbić go na prostsze problemy. W trudniejszych przypadkach właśnie taka dekompozycja może być bardziej naturalna niż dopisywanie kolejnych linii od początku dowodu.

To prowadzi do trzeciej strategii, w której system może najpierw rozbić dowodzony sekwent na listę prostszych podproblemów, a następnie próbować udowodnić każdy z nich osobno.

3.4. Strategia trzecia: dekompozycja celu i drzewo poszukiwania

3.4.1. Motywacja

Druga strategia pozwala prowadzić wiele częściowych dowodów równolegle, dzięki czemu system nie jest przywiązany do jednej liniowej ścieżki generacji. Nadal jednak rozwijanie dowodu odbywa się przez dopisywanie kolejnych linii do już istniejącego fragmentu. System wybiera więc pewien częściowy dowód i próbuje znaleźć jego następny poprawny krok. Nie analizuje natomiast osobno, jaka ogólna struktura dowodu byłaby potrzebna, aby otrzymać zadany sekwent.

Taki sposób działania odpowiada naturalnemu zapisowi dowodu w notacji Fitcha: zaczynamy od założeń, aksjomatów lub wcześniej uzyskanych sekwentów, a następnie stosujemy reguły wnioskowania, aby dojść do tezy. Jest to konstruowanie dowodu w kierunku od przesłanek do celu. W wielu sytuacjach wygodny jest jednak kierunek przeciwny. Zamiast pytać, jaką kolejną linię można dopisać do częściowego dowodu, można zacząć od sekwentu, który chcemy udowodnić, i zapytać, jakie prostsze sekwenty wystarczyłoby udowodnić, aby otrzymać go przez zastosowanie jednej z reguł wnioskowania.

Można więc patrzeć na konstrukcję dowodu z dwóch stron. Pierwsza strategia buduje dowód od wcześniejszych linii ku końcowej tezie. Trzecia strategia zaczyna od celu i próbuje rozłożyć go na podcele, które następnie mogą być dowodzone osobno. W notacji drzewiastej odpowiada to budowaniu drzewa dowodu od korzenia ku liściom: w korzeniu znajduje się sekwent, który chcemy udowodnić, a kolejne poziomy drzewa zawierają przesłanki reguł, których udowodnienie wystarczy do otrzymania

sekwentów znajdujących się wyżej.

Przykładowo, jeżeli celem jest udowodnienie koniunkcji, naturalnym krokiem jest próba udowodnienia osobno jej dwóch składników. Jeżeli celem jest udowodnienie implikacji, można przyjąć jej poprzednik jako dodatkowe założenie i następnie spróbować udowodnić jej następnik. W obu przypadkach nie zaczynamy od pytania o pierwszą linię dowodu, lecz od pytania o to, jakie podproblemy prowadzą do rozwiązania problemu wejściowego.

Trzecia strategia rozszerza więc wcześniejsze podejścia o możliwość dekompozycji celu. System może najpierw rozbić dowodzony sekwent na listę prostszych podproblemów, a dopiero potem próbować udowodnić każdy z nich za pomocą wcześniejszych metod generowania dowodów. Jeżeli wszystkie podproblemy danego rozbicia zostaną rozwiązane, to rozwiązany zostaje również problem wejściowy.

3.4.2. Rozbicie problemu na podproblemy

Rozbiciem problemu na podproblemy nazywamy wskazanie takiej reguły wnioskowania oraz takiej listy sekwentów-przesłanek, że sekwent wejściowy można otrzymać przez zastosowanie tej reguły do tych przesłanek.

Aby zapisać to precyzyjniej, dla każdej reguły R oraz listy sekwentów $S_1, \dots, S_n$ wprowadzamy oznaczenie

$$R(S_1, \dots, S_n),$$

które oznacza zbiór wszystkich sekwentów mogących wystąpić jako konkluzje poprawnego zastosowania reguły R do przesłanek $S_1, \dots, S_n$.

Jeżeli więc chcemy udowodnić sekwent S , to szukamy reguły R oraz sekwentów $S_1, \dots, S_n$, dla których zachodzi

$$S \in R(S_1, \dots, S_n).$$

Wtedy zadanie udowodnienia S można zastąpić zadaniami udowodnienia $S_1, \dots, S_n$.

Takie rozbicie można interpretować jako jeden krok budowania drzewa dowodu w kierunku przeciwnym do kierunku zapisu dowodu w notacji Fitcha. W notacji Fitcha zaczynamy zwykle od prostych przesłanek i dopisujemy kolejne linie aż do uzyskania celu. W dekompozycji celu zaczynamy natomiast od sekwentu, który chcemy otrzymać, i pytamy, jakie przesłanki mogłyby prowadzić do niego przez jedną z reguł wnioskowania.

Schematycznie:

$$\begin{array}{ccccc} \text{problem} & & \text{wybór} & & \text{podproblemy} \\ S & \longrightarrow & \text{reguły } R & \longrightarrow & S_1, \dots, S_n \end{array}$$

przy czym warunkiem poprawności rozbicia jest

$$S \in R(S_1, \dots, S_n).$$

Rozbicie z założenia powinny generować podproblemy o dowodach prostszych niż dowód problemu początkowego. Co więcej trzecia strategia nie zastępuje generowania dowodów liniowych, ale organizuje je w większą strukturę: zamiast od razu znaleźć jeden pełny dowód sekwentu, system może rozbić go na mniejsze sekwenty i dla każdego z nich uruchomić szukanie dowodu w formie liniowej. Szukanie to odbywa się przy pomocy strategii drugiej.

3.4.3. Format zapytania o rozbicie

W trzeciej strategii model językowy jest używany nie tylko do generowania kolejnych linii dowodu, lecz także do proponowania rozbić celu na podproblemy. Jest to osobne zadanie generatywne. W przypadku dowodzenia liniowego model otrzymuje zapis celu oraz dotychczasowy fragment dowodu i ma zaproponować jego dalszy ciąg. W przypadku rozbijania problemu model otrzymuje sekwent, który chcemy udowodnić, i ma wskazać regułę wnioskowania oraz listę podproblemów, których udowodnienie pozwoli otrzymać sekwent wejściowy.

Z tego powodu w systemie wykorzystywane są dwa modele językowe o tej samej architekturze transformera, ale trenowane na różnych typach danych. Pierwszy model uczy się generować dowody w notacji tekstowej, czyli dopisywać kolejne linie częściowego dowodu. Drugi model uczy się generować rozbitcia problemów na podproblemy. Oba modele korzystają z podobnej idei: proponują tekstowy kandydat, który następnie jest sprawdzany przez formalny parser i mechanizm weryfikacji poprawności. Różnią się natomiast formatem danych treningowych oraz rodzajem tekstu, który mają wygenerować.

Aby drugi model mógł być trenowany i używany w taki sam sposób jak model dowodzący, rozbitcia muszą zostać zapisane w jednoznacznym formacie tekstowym. Format ten powinien określać trzy informacje: sekwent, który jest rozbijany, regułę wnioskowania używaną do rozbitcia oraz listę sekwentów-przesłanek otrzymanych po zastosowaniu tej reguły.

W systemie rozbitcie zapisywane jest jako tekst o następującym formacie:

Smash: S
 With: R
 To: S_1 ,
 ...
 S_n

gdzie S jest sekwentem, który chcemy rozbić, R jest nazwą reguły wnioskowania, a $S_1, \dots, S_n$ są sekwentami-przesłankami otrzymanymi po rozbiciu.

Pierwsza linia określa problem wejściowy. Druga linia wskazuje regułę, za pomocą której problem ma zostać rozbit. Kolejne linie zawierają listę podproblemów.

Jeżeli reguła nie wymaga żadnych przesłanek (np reguła **Assumption**), lista podproblemów jest pusta.

Przykładowy schemat rozbicia można zapisać następująco:

Smash: $\Gamma \vdash \varphi \rightarrow \psi$
With: $\rightarrow$ -introduction
To: $\Gamma, \varphi \vdash \psi$

Taki zapis oznacza, że problem udowodnienia implikacji został zastąpiony problemem udowodnienia jej prawej strony przy dodatkowym założeniu jej lewej strony.

3.4.4. Sprawdzanie poprawności rozbicia

Ponieważ rozbicia są generowane przez model językowy, każde wygenerowane rozbicie musi zostać sprawdzone formalnie. Rozbicie składa się z problemu wejściowego S , reguły R oraz listy podproblemów $S_1, \dots, S_n$. Jest poprawne wtedy, gdy zastosowanie reguły R do podproblemów pozwala otrzymać problem wejściowy.

W naszej implementacji reguły wnioskowania mogą być stosowane tylko do sekwentów, które mają ten sam kontekst bazowy. Dlatego przed sprawdzeniem rozbicia wszystkie podproblemy sprowadzane są do postaci z pustym kontekstem bazowym. Cały ich kontekst logiczny zostaje przeniesiony do kontekstu dodatkowego. Jeżeli podproblem ma kontekst bazowy Γ_b , kontekst dodatkowy Γ_d i tezę φ , to zastępujemy go sekwentem

$$\emptyset; \Gamma_b, \Gamma_d \vdash \varphi.$$

Gdzie średnik oddziela tutaj kontekst bazowy od kontekstu dodatkowego.

Tak samo przekształcany jest problem wejściowy S . Nie jest to potrzebne do samego zastosowania reguły, ale upraszcza końcowe porównanie: wynik zastosowania reguły do przekształconych podproblemów porównujemy z problemem wejściowym zapisanym w tej samej konwencji.

Procedura sprawdzania wygląda więc następująco:

Algorithm 4 Sprawdzanie poprawności rozbicia

```

1: procedure SPRAWDŹROZBICIE( $S, R, S_1, \dots, S_n$ )
2:    $S' \leftarrow$  sekwent  $S$  sprowadzony do postaci z pustym kontekstem bazowym
3:   for  $i = 1, \dots, n$  do
4:      $S'_i \leftarrow$  sekwent  $S_i$  sprowadzony do postaci z pustym kontekstem bazowym
5:     if  $S_i$  nie jest sekwentem tautologicznym then
6:       return odrzuć rozbicie
7:     end if
8:   end for
9:    $A \leftarrow$  dodatkowe argumenty potrzebne do zastosowania reguły  $R$ 
10:  spróbuj zastosować  $R$  do  $S'_1, \dots, S'_n$  z argumentami  $A$ 
11:  if zastosowanie reguły zgłasza błąd then
12:    return odrzuć rozbicie
13:  end if
14:   $T \leftarrow$  wynik zastosowania reguły  $R$ 
15:  if  $T \neq S'$  then
16:    return odrzuć rozbicie
17:  end if
18:  return zaakceptuj rozbicie
19: end procedure

```

Dodatkowe argumenty podobnie jak przy przetwarzaniu dowodów liniowych są potrzebne dla tych reguł, których zastosowanie nie jest jednoznacznie wyznaczone przez same przesłanki. W takich przypadkach brakujące dane są dedukowane z problemu wejściowego lub z problemu wejściowego i przesłanek.

Sprawdzanie tautologiczności podproblemów pełni w systemie rolę pomocniczego filtra. Nie jest ono potrzebne do samego sprawdzenia, czy dane rozbicie odpowiada schematowi wybranej reguły wnioskowania. Pozwala jednak od razu odrzucić takie rozbicia, które prowadzą do podproblemów niedowodliwych, a więc do gałęzi drzewa poszukiwania, które nie mogłyby zostać domknięte.

Rozwiązanie to jest mało eleganckie z punktu widzenia czysto dowodowego, ponieważ system sprawdza semantycznie, czy sekwent, dla którego następnie ma znaleźć dowód, jest tautologiczny. W niniejszej pracy jest to jednak świadome uproszczenie implementacyjne. System ma charakter prototypowy i eksperymentalny, a rozważane formuły są na tyle małe, że sprawdzenie wszystkich wartościowań nie stanowi istotnego kosztu obliczeniowego. Dzięki temu można ograniczyć liczbę oczywiście bezowocnych prób i skupić się na właściwym celu pracy, czyli generowaniu formalnych dowodów przez model językowy.

Należy podkreślić, że filtr tautologiczności nie zastępuje dowodu. Zaakceptowany podproblem nadal musi zostać udowodniony w przyjętej notacji, a każda linia wygenerowanego dowodu jest następnie sprawdzana przez parser i implementację

reguł wnioskowania. Sprawdzenie tautologiczności pełni więc jedynie funkcję praktycznego ograniczenia przestrzeni poszukiwań, a nie mechanizmu konstruowania dowodu.

3.4.5. Generowanie rozbić przez model

Generowanie rozbitcia przebiega analogicznie do generowania kolejnych linii dowodu: model językowy nie tworzy od razu struktury logicznej, lecz dopisuje tekst w ustalonym formacie. Punktem wyjścia jest prompt zawierający sekwent, który ma zostać rozbitý:

Smash: S

Na podstawie tego promptu model najpierw generuje linię wskazującą regułę wnioskowania, czyli linię zaczynającą się od *With:*. Następnie generowana jest linia *To:*, zawierająca listę podproblemów, na które ma zostać rozbitý sekwent wejściowy.

Podczas generowania podproblemów stosowane jest proste ograniczenie składowe: model może dopisywać jedynie symbole logiczne, separatory oraz zmienne występujące w rozbijanym sekwencie. Dzięki temu zmniejsza się liczba kandydatów, które już na poziomie zapisu wprowadzają zupełnie nowe zmienne i z tego powodu nie mogą być sensownymi rozbiciami danego problemu. Nie jest to jednak pełna kontrola poprawności. Wygenerowany tekst nadal musi zostać sparsowany i sprawdzony przez procedurę weryfikującą rozbitcie.

Model może też wygenerować listę podproblemów w kilku liniach. Przyjmujemy, że opis rozbitcia jest gotowy do sprawdzenia wtedy, gdy ma już co najmniej trzy linie oraz ostatnia linia nie kończy się przecinkiem. Przecinek na końcu oznacza, że model sugeruje dalszy ciąg listy podproblemów, więc generowanie powinno być kontynuowane.

Schemat działania można zapisać następująco:

Algorithm 5 Generowanie rozbitcia

```

1: procedure WYGENERUJROZBICIE( $S, N$ )
2:    $p_0 \leftarrow \text{Smash}: S$ 
3:    $p \leftarrow p_0$ 
4:   for  $i = 1, \dots, N$  do
5:     while  $p$  nie jest gotowe do sprawdzenia do
6:       wygeneruj następną linię tekstu na podstawie  $p$ 
7:       dopisz wygenerowaną linię do  $p$ 
8:     end while
9:     if  $p$  koduje poprawne rozbitcie then
10:      return  $p$ 
11:     else
12:        $p \leftarrow p_0$ 
13:     end if
14:   end for
15:   return informacja o niepowodzeniu
16: end procedure

```

Jeżeli wygenerowany tekst nie przechodzi weryfikacji, zostaje odrzucony w całości poza pierwszą linią $\text{Smash}:S$, która określa sekwent do rozbitcia. System nie próbuje naprawiać błędnego kandydata lokalnie, ponieważ błąd może wynikać zarówno z wyboru niewłaściwej reguły, jak i z niepoprawnej listy podproblemów albo z samego formatu tekstowego. W takiej sytuacji prostsze i bardziej stabilne jest ponowienie generacji od tego samego punktu początkowego. Procedura powtarza ten proces do momentu znalezienia poprawnego rozbitcia albo osiągnięcia ustalonego limitu prób.

3.4.6. Dowodzenie sekwentów z założeniami

Dotychczas najprostsza wersja systemu mogła być rozumiana jako narzędzie do dowodzenia tautologii, czyli sekwentów o pustym kontekście. Wprowadzenie rozbić problemów na podproblemy zmienia jednak tę sytuację. Nawet jeśli problem wyjściowy ma postać

$$\vdash \varphi,$$

to jego podproblemy mogą mieć już niepuste konteksty. Przykładowo rozbitcie celu

$$\vdash \varphi \rightarrow \psi$$

przez regułę wprowadzania implikacji prowadzi do podproblemu

$$\varphi \vdash \psi.$$

Oznacza to, że system musi umieć nie tylko dowodzić tautologii, lecz także generować dowody sekwentów z założeniami.

Przy dowodzeniu sekwentów z założeniami pojawia się dodatkowa trudność: należy kontrolować, które formuły należą do kontekstu sekwentu kodowanego przez daną linię dowodu. Naturalnym uproszczeniem jest przyjęcie, że założenia sekwentu wejściowego są dostępne przez cały czas trwania dowodu, a więc znajdują się w kontekście każdej generowanej linii.

To uproszczenie prowadzi do podziału kontekstu na dwie części. Kontekst bazowy, oznaczany w implementacji jako `base_context`, zawiera formuły traktowane jako stale dostępne w całym rozważanym dowodzie. Są to założenia dowodzonego sekwentu, zapisane w nagłówku promptu. Jako `additional_context` z kolei oznaczamy kontekst dodatkowy, który zawiera natomiast założenia lokalne, które mogą zmieniać się pomiędzy liniami dowodu.

Reguły wnioskowania stosujemy wyłącznie do przesłanek mających ten sam kontekst bazowy oraz wynik działania reguły ma również ten sam kontekst bazowy co przesłanki.

Działanie reguł wnioskowania dla przesłanek z kontekstem podzielonym na kontekst bazowy i dodatkowy można rozumieć jako zastosowanie tych reguł w klasyczny sposób, czyli bez rozróżnienia na kontekst bazowy i dodatkowy, a następnie uzupełnieniu kontekstu konkluzji brakującymi formułami należącymi do kontekstu bazowego.

Można więc powiedzieć, że schemat działania reguły R dla sekwentów z rozróżnionym kontekstem bazowym i dodatkowym polega na zastąpieniu schematu

$$\frac{\Gamma_1 \vdash \varphi_1 \quad \Gamma_2 \vdash \varphi_2 \quad \cdots \quad \Gamma_n \vdash \varphi_n}{\Delta \vdash \psi} (R)$$

schematem:

$$\frac{\Gamma; \Gamma_1 \setminus \Gamma \vdash \varphi_1 \quad \Gamma; \Gamma_2 \setminus \Gamma \vdash \varphi_2 \quad \cdots \quad \Gamma; \Gamma_n \setminus \Gamma \vdash \varphi_n}{\Gamma; \Delta \setminus \Gamma \vdash \psi} (R)$$

Zakładamy, że kontekst bazowy i dodatkowy każdego sekwentu są rozłączne.

Taka konwencja wymaga jednak uzasadnienia poprawności. Po pierwsze, trzeba pokazać, że jeśli dany sekwent można udowodnić w zwykłej reprezentacji, czyli bez wyróżnionego kontekstu bazowego, to można go również udowodnić w reprezentacji, w której wszystkie sekwenty pośrednie mają jako kontekst bazowy założenia problemu wejściowego.

Dowód tej zależności jest bardzo prosty, ponieważ ze zdefiniowania działania reguł dla sekwentów z kontekstem bazowym wynika wprost sposób konwersji dowodu z jednej reprezentacji na drugą. Z kolei sekwent w samym korzeniu drzewa dowodu po takiej konwersji ma pusty kontekst dodatkowy (bo $\Gamma; \Delta \setminus \Gamma \vdash \psi$ przybiera postać $\Gamma; \Gamma \setminus \Gamma \vdash \psi$, czyli $\Gamma; \emptyset \vdash \psi$).

Po drugie, trzeba pokazać zależność odwrotną: użycie kontekstu bazowego nie może pozwalać na udowodnienie sekwentu, którego nie dałoby się uzyskać w zwykłej reprezentacji. Jeśli więc istnieje dowód, w którym sekwenty pośrednie korzystają z niepustego kontekstu bazowego, to powinien istnieć również odpowiadający mu dowód zapisany bez tego technicznego rozróżnienia, czyli taki, w którym przesłanki mają kontekst bazowy pusty.

Dowód tej zależności będzie polegać na pokazaniu algorytmu, który dowód dla sekwentów z kontekstem bazowym równym kontekstowi bazowemu tezy konwertuje na dowód z sekwentami o pustym kontekście bazowym.

Dla uproszczenia rozważań przyjmujemy konwencję jest w której istnieje ukryta linia o indeksie 0. Nie jest ona wypisywana w tekście dowodu, a numeracja jawnych linii zaczyna się od 1. Linia 0 koduje sekwent, którego kontekstem jest przyjęty kontekst bazowy, a tezą jest $\top$. Dzięki temu reguły **context** oraz **contextWeak** mogą odwoływać się do całego kontekstu bazowego bez konieczności dopisywania osobnej linii dowodu której jedyną funkcją jest posiadanie kontekstu równego kontekstowi bazowemu. Jeżeli kontekst bazowy jest pusty, linia 0 koduje sekwent $\emptyset \vdash \top$.

Idea algorytmu bazuje na następującej obserwacji: jeżeli poprawny jest schemat

$$\frac{\Gamma; \Gamma'_1 \vdash \varphi_1 \quad \Gamma; \Gamma'_2 \vdash \varphi_2 \quad \cdots \quad \Gamma; \Gamma'_n \vdash \varphi_n}{\Gamma; \Delta' \vdash \psi} (R)$$

To z definicji działania reguł dla sekwentów z kontekstem bazowym wynika, że istnieją takie zbiory formuł $\Gamma_1, \dots, \Gamma_n$ oraz Δ , że $\Gamma'_i = \Gamma_i \setminus \Gamma$ dla $i = 1, \dots, n$ oraz $\Delta' = \Delta \setminus \Gamma$ oraz schemat

$$\frac{\Gamma_1 \vdash \varphi_1 \quad \Gamma_2 \vdash \varphi_2 \quad \cdots \quad \Gamma_n \vdash \varphi_n}{\Delta \vdash \psi} (R)$$

Jednak ze schematu każdej reguły wnioskowania w naszym systemie, która ma niepustą listę przesłanek, wynika, że jeżeli dla przesłanek $S_1, \dots, S_n$ można zastosować regułę R można ją również zastosować do tych samych przesłanek, ale z rozszerzonymi o jakieś formuły kontekstami, a co za tym idzie w rozważanym przypadku

$$\frac{\Gamma_1 \cup \Gamma \vdash \varphi_1 \quad \Gamma_2 \cup \Gamma \vdash \varphi_2 \quad \cdots \quad \Gamma_n \cup \Gamma \vdash \varphi_n}{\Gamma' \vdash \psi} (R)$$

dla jakiegoś Γ' zawierającego się w $\Gamma \cup \Delta$.

Algorytm usuwania kontekstu bazowego działa następująco. Niech

$$\Gamma = \{\gamma_1, \dots, \gamma_m\}$$

będzie kontekstem bazowym dowodu wejściowego. Na początku nowego dowodu wprowadzamy kolejno wszystkie formuły z Γ jako jawne linie dowodu. Robimy to

przy użyciu reguły **contextWeak**. Pierwsza z tych linii odwołuje się do ukrytej linii o indeksie 0, a każda kolejna do poprzednio uzyskanej linii, dzięki czemu po m krokach otrzymujemy linię, której kontekst dodatkowy jest równy całemu dawnemu kontekstowi bazowemu (można to zrobić bo linii o indeksie 0 używamy tylko w regule **context** i **contextWeak**, dla których nie jest istotna konkluzja przesłanki).

Następnie algorytm przechodzi przez kolejne linie oryginalnego dowodu, ale nie kopiuje ich dosłownie. Dla każdej starej linii odczytywana jest reguła, która została w niej użyta, oraz indeksy wcześniejszych linii, do których ta reguła się odwołuje. Ponieważ w nowym dowodzie pojawiają się dodatkowe linie, stare indeksy nie mogą zostać użyte bezpośrednio. Algorytm przechowuje więc mapę indeksów, która każdej linii starego dowodu przypisuje linię odpowiadającą jej w nowym dowodzie.

Dla rozważanej starej linii algorytm bierze indeksy jej przesłanek, zastępuje je indeksami wskazanymi przez tę mapę, a następnie stosuje tę samą regułę do odpowiadających im linii nowego dowodu. Otrzymana w ten sposób linia nie jest kopią starej linii, lecz jej odpowiednikiem skonstruowanym już w dowodzie bez kontekstu bazowego. Jeżeli stara linia odwoływała się do ukrytej linii o indeksie 0, to odwołanie to zostaje zastąpione odwołaniem do linii o indeksie m w nowym dowodzie, czyli do linii, której kontekst dodatkowy jest równy całemu dawnemu kontekstowi bazowemu (można to zrobić bo linii o indeksie 0 używamy tylko w regule **context** i **contextWeak**, dla których nie jest istotna konkluzja przesłanki).

Po skonstruowaniu takiego odpowiednika algorytm sprawdza, czy jego kontekst dodatkowy zawiera wszystkie formuły z dawnego kontekstu bazowego. Jeżeli którejś z nich brakuje, dodawane są kolejne linie otrzymane przez zastosowanie reguły **weakening**. Dopiero ostatnia linia uzyskana po tych ewentualnych osłabieniach jest traktowana jako właściwy odpowiednik danej linii starego dowodu i jej numer zostaje zapisany w mapie indeksów.

Dla skrócenia otrzymanego dowodu, jeżeli przetwarzana stara linia była linią z regułą **assumption**, to w nowym dowodzie stosowana jest reguła **contextWeak** odwołująca się do linii o indeksie m .

Schematycznie procedurę można zapisać następująco:

Algorithm 6 Usuwanie kontekstu bazowego

```

1: procedure USUŃKONTEKSTBAZOWY( $D$ )
2:    $\Gamma = \{\gamma_1, \dots, \gamma_m\} \leftarrow$  kontekst bazowy dowodu  $D$ 
3:    $D' \leftarrow$  pusty dowód
4:    $f(0) \leftarrow 0$ 
5:   for  $i = 1, \dots, m$  do
6:     dopisz do  $D'$  linię wyprowadzającą  $\gamma_i$  za pomocą contextWeak,  $i-1$ 
7:   end for
8:    $f(0) \leftarrow m$ 
9:   for każdą jawną linię  $l_i$  dowodu  $D$  do
10:    odczytaj z  $l_i$  regułę  $R$ , formułę  $\varphi_i$  oraz indeksy  $j_1, \dots, j_n$ 
11:    zamień indeksy przesłanek na  $f(j_1), \dots, f(j_n)$ 
12:    if  $R = \text{assumption}$  then
13:      dopisz do  $D'$  wynik zastosowania contextWeak do linii o indeksie  $f(0)$ 
14:    end if
15:    if  $R \neq \text{assumption}$  then
16:      dopisz do  $D'$  wynik zastosowania  $R$  do linii  $f(j_1), \dots, f(j_n)$ 
17:      for  $j \in 1, \dots, m$  do
18:        if  $\gamma_j$  nie należy do kontekstu ostatniej linii  $D'$  then
19:           $n \leftarrow$  liczba linii  $D'$ 
20:           $l = n+1$ .  $\phi_i$  weakening  $j$ ,  $n$ 
21:          dopisz  $l$  do  $D'$ 
22:        end if
23:      end for
24:    end if
25:     $f(i) \leftarrow$  numer ostatniej linii dowodu  $D'$ 
26:  end for
27:  return  $D'$ 
28: end procedure

```

3.4.7. Drzewo poszukiwań

W tej części pracy opisujemy główny algorytm strategii trzeciej, oraz strukturę danych, na której jest ona oparta.

Aby połączyć dekompozycję celu z generowaniem dowodów częściowych, system buduje drzewo poszukiwania. Korzeniem tego drzewa jest sekwent, który chcemy udowodnić. Następnie system może wygenerować kilka różnych rozbić tego sekwentu na podproblemy. Każde takie rozbiecie stanowi osobną możliwość kontynuacji poszukiwania dowodu. Dzięki temu system nie musi od razu decydować się na jedną postać rozumowania, lecz może przechowywać równolegle kilka alternatywnych struktur dowodu.

W drzewie występują dwa typy wierzchołków. Pierwszy typ reprezentuje problemy dowodowe, czyli sekwenty, które należy udowodnić. Będziemy oznaczać je jako wierzchołki typu LP. Drugi typ reprezentuje konkretne rozbiecia problemu na podproblemy. Będziemy oznaczać je jako wierzchołki typu SP.

Dzieci wierzchołka typu LP są wierzchołkami typu SP, kodującymi możliwe rozbiecia sekwentu zapisanego w ich rodzicu. Z kolei dzieci wierzchołka typu SP są wierzchołkami typu LP, które kodują sekwent będące przesłankami rozbiecia zapisanego w ich rodzicu. Struktura drzewa jest więc naprzemienna: dziećmi wierzchołków problemów dowodowych są wierzchołki rozbić, a dziećmi wierzchołków rozbić są wierzchołki problemów dowodowych.

Intuicyjnie działanie algorytmu polega na wielokrotnym wybieraniu nierozwiązanych fragmentów drzewa i próbie ich dowiedzenia.

Jeżeli próbujemy udowodnić wierzchołek typu LP, który jest liściem, system najpierw próbuje znaleźć jego bezpośredni dowód za pomocą strategii drugiej. Jeśli to się uda, dowód zostaje zapisany w wierzchołku, a wartość wierzchołka zmienia się na prawdziwą. Jeżeli bezpośredni dowód nie zostanie znaleziony, system generuje możliwe rozbiecia tego sekwentu i dodaje je jako dzieci wierzchołka. Następnie jako dzieci każdego z tych dzieci dodawane są wierzchołki typu LP kodujące przesłanki rozbiecia zapisanego w ich rodzicu. Jeżeli któryś z wierzchołków rozbiecia nie ma żadnych przesłanek, zostaje od razu oznaczony jako udowodniony.

Wierzchołek typu SP reprezentuje jedno konkretne rozbiecie. Jego dziećmi są sekwenty-przesłanki tego rozbiecia. Aby udowodnić rozbiecie, trzeba udowodnić wszystkie te sekwenty. Dlatego dla wierzchołka typu SP algorytm próbuje rekurencyjnie dowieść wszystkich jego dzieci. Jeśli choć jedno z nich nie zostanie udowodnione w danej próbie, całe rozbiecie nie zostaje uznane za udowodnione.

W przypadku wewnętrznego wierzchołka typu LP sytuacja jest inna: wystarczy, że powiedzie się próba udowodnienia jednego z jego rozbić. Dlatego algorytm wybiera jedno z jego dzieci typu SP i próbuje je udowodnić. Wybór ten odbywa się z wagami wprost proporcjonalnymi do częstości, z jaką dane rozbiecie zostało wygenerowane przez model. Jeżeli wybrane rozbiecie zostanie udowodnione, to udowodniony zostaje również sekwent reprezentowany przez jego rodzica.

Po każdej udanej próbie dowodu wierzchołka informacja o tym które wierzchołki zostały udowodnione jest propagowana w górę drzewa. Jeżeli wierzchołek typu SP zostaje udowodniony, jego rodzic typu LP jest oznaczany jako udowodniony, a informacja o tym jest przekazywana dalej do jego rodzica, aż do korzenia drzewa. Jeżeli zaś udowodniony zostaje wierzchołek typu LP, to dla jego rodzica typu SP sprawdzamy, czy wszystkie jego dzieci są już udowodnione. Jeżeli tak, to również ten wierzchołek typu SP jest oznaczany jako udowodniony, a informacja o tym jest propagowana dalej dla kolejnych przodków, aż do korzenia.

Opisany mechanizm można wygodnie podzielić na cztery procedury. Pierwsza

z nich odpowiada za obsługę liścia typu LP, czyli sekwentu, dla którego nie znamy jeszcze ani bezpośredniego dowodu, ani rozbicia na podproblemy. Druga procedura odpowiada za samo wygenerowanie rozbić takiego liścia i dołączenie ich do drzewa. Trzecia procedura opisuje ogólne rekurencyjne przeszukiwanie drzewa, korzystające z dwóch poprzednich procedur jako podprogramów. Czwarta procedura zaś uruchamia w pętli procedurę trzecią poszukując dowodu w drzewie po kolejnych aktualizacjach jego struktury.

Najpierw rozważmy obsługę liścia typu LP. Taki wierzchołek reprezentuje pojedynczy sekwent, który system próbuje udowodnić. Naturalnie najpierw sprawdzamy, czy da się znaleźć jego dowód bez dalszego rozbijania problemu. W tym celu uruchamiana jest strategia druga szukania dowodu. Jeżeli zakończy się ona sukcesem, znaleziony dowód zostaje zapisany w wierzchołku, a sam wierzchołek zostaje oznaczony jako udowodniony. Jeżeli natomiast dowodu bezpośredniego nie uda się znaleźć w przyjętych limitach, wierzchołek zostaje rozbity na podproblemy.

Algorithm 7 Obsługa liścia typu LP

```

1: procedure OBSLUZLISC( $X$ )
2:   spróbuj znaleźć bezpośredni dowód sekwentu kodowanego przez  $X$ 
3:   if dowód został znaleziony then
4:     zapisz znaleziony dowód w  $X$ 
5:     oznacz  $X$  jako udowodniony
6:     zaktualizuj wartości przodków  $X$ 
7:     return True
8:   else
9:     ROZBIJLISC( $X$ )
10:    return False
11:  end if
12: end procedure

```

Jeżeli nie uda się znaleźć bezpośredniego dowodu sekwentu zapisanego w liściu, procedura obsługi liścia deleguje rozbicie tego wierzchołka do kolejnej procedury. Procedura rozbijania nie wybiera zwykle pojedynczego rozbicia, lecz wykonuje określoną liczbę prób wygenerowania rozbicia przez model. Liczbę tych prób oznaczamy przez N_{roz} . Każde poprawne rozbicie jest zapamiętywane, a jeśli to samo rozbicie pojawi się wielokrotnie (co może zdarzać się dość często, bo często jest bardzo mało naturalnych rozbić), zwiększana jest jego waga używana w kolejnej procedurze do losowania obsługiwanego wierzchołka. Dzięki temu powtarzające się propozycje modelu nie są traktowane jako zbędne duplikaty, lecz jako informacja o tym, które rozbicia model uznaje za bardziej naturalne.

Po zakończeniu generowania poprawne rozbicia są dodawane jako dzieci rozważanego wierzchołka typu LP. Następnie dla każdego wierzchołka typu SP dodawane są jego dzieci typu LP, kodujące sekwent-przesłanki danego rozbicia. Jeżeli dane

rozbicie nie ma żadnych przesłanek, odpowiadający mu wierzchołek typu SP zostaje od razu oznaczony jako udowodniony.

Algorithm 8 Rozbijanie liścia typu LP

```

1: procedure ROZBIJLISC( $X, N_{\text{roz}}$ )
2:   utwórz prompt opisujący sekwent kodowany przez  $X$ 
3:    $W \leftarrow$  pusty słownik wag rozbić
4:   for  $i = 1, \dots, N_{\text{roz}}$  do
5:     wygeneruj kandydackie rozbicie  $S$ 
6:     if  $S$  jest poprawnym rozbiem then
7:       zwiększ wagę rozbięcia  $S$  w słowniku  $W$ 
8:     end if
9:   end for
10:  for każde rozbicie  $S$  zapisane w  $W$  do
11:    dodaj do  $X$  dziecko typu SP kodujące rozbicie  $S$ 
12:    przypisz temu dziecku wagę proporcjonalną do częstości  $S$ 
13:  end for
14:  for każde dziecko  $Y$  wierzchołka  $X$  do
15:    for każda przesłanka  $P$  rozbięcia zapisanego w  $Y$  do
16:      dodaj do  $Y$  dziecko typu LP kodujące sekwent  $P$ 
17:    end for
18:    if  $Y$  nie ma dzieci then
19:      oznacz  $Y$  jako udowodniony
20:      zaktualizuj wartości przodków  $Y$ 
21:    end if
22:  end for
23: end procedure

```

Trzecia procedura opisuje właściwe przeszukiwanie drzewa. Jeżeli aktualny wierzchołek jest już oznaczony jako udowodniony, algorytm od razu kończy pracę sukcesem. Jeżeli jest liściem typu LP, uruchamiana jest opisana wyżej procedura obsługi liścia. Jeżeli jest wewnętrznym wierzchołkiem typu LP, algorytm wybiera jedno z jego dzieci typu SP zgodnie z wagami rozbić i próbuje udowodnić właśnie tę gałąź. Jeżeli natomiast aktualny wierzchołek jest typu SP, to algorytm musi udowodnić wszystkie jego dzieci, ponieważ dane rozbicie jest skuteczne dopiero wtedy, gdy rozwiązane są wszystkie jego podproblemy.

Algorithm 9 Przeszukiwanie drzewa dowodu

```

1: procedure DOWIEDZ( $X$ )
2:   if  $X$  jest już oznaczony jako udowodniony then
3:     return True
4:   end if
5:   if  $X$  jest liściem typu LP then
6:     return OBSLUZLISC( $X$ )
7:   end if
8:   if  $X$  jest wewnętrznym wierzchołkiem typu LP then
9:     wybierz jedno z dzieci  $X$ , oznaczmy je przez  $Y$ 
10:    wybór wykonaj z prawdopodobieństwami zapisanymi w dzieciach  $X$ 
11:    return DOWIEDZ( $Y$ )
12:  end if
13:  if  $X$  jest wierzchołkiem typu SP then
14:    for każde dziecko  $Y$  wierzchołka  $X$  do
15:       $b \leftarrow$  DOWIEDZ( $Y$ )
16:      if  $b = \text{False}$  then
17:        return False
18:      end if
19:    end for
20:    oznacz  $X$  jako udowodniony
21:    zaktualizuj wartości przodków  $X$ 
22:    return True
23:  end if
24: end procedure

```

Główna procedura poszukiwania dowodu uruchamia procedurę DOWIEDZ w pętli. Pojedyncze wywołanie nie musi od razu znaleźć całego dowodu, a może jedynie rozbudować drzewo lub oznaczyć część wierzchołków jako udowodnione, dlatego kolejne wywołania korzystają z coraz bardziej rozwiniętej struktury. Działanie kończy się sukcesem, jeżeli korzeń drzewa zostanie oznaczony jako udowodniony. Wtedy system odtwarza z drzewa pełny dowód tekstowy. Jeżeli nie uda się tego zrobić po $N_{\max}$ próbach, procedura kończy się niepowodzeniem.

Algorithm 10 Główna pętla poszukiwania dowodu

```

1: procedure SZUKAJDOWODU( $G, N_{\max}$ )
2:   utwórz korzeń  $K$  typu LP kodujący cel  $G$ 
3:   for  $i = 1, \dots, N_{\max}$  do
4:     if  $K$  jest oznaczony jako udowodniony then
5:       return ODTWORZDOWOD( $K$ )
6:     end if
7:      $b \leftarrow$  DOWIEDZ( $K$ )
8:     if  $b = \text{True}$  albo  $K$  jest oznaczony jako udowodniony then
9:       return ODTWORZDOWOD( $K$ )
10:    end if
11:  end for
12:  return False
13: end procedure

```

3.4.8. Sklejanie dowodów

Samo znalezienie rozwiązania w drzewie poszukiwania nie oznacza jeszcze, że mamy gotowy tekstowy dowód pierwotnej tezy. Drzewo zawiera informację o tym, które podproblemy zostały udowodnione oraz przez jakie rozbicia można je połączyć, ale sam dowód jest rozproszony po różnych wierzchołkach.

Procedura sklejania działa rekurencyjnie od liści drzewa ku korzeniowi. Jeżeli rozwiązany wierzchołek typu LP jest liściem, to zawiera bezpośrednio wygenerowany dowód danego sekwentu. Taki dowód może mieć niepusty kontekst bazowy, dlatego najpierw stosowana jest opisana wcześniej procedura usuwania kontekstu bazowego. W efekcie otrzymujemy zwykły fragment dowodu, który można wkleić do większego dowodu.

Jeżeli natomiast wierzchołek typu LP nie jest liściem, to znaczy, że został rozwiązany przez jedno ze swoich rozbić. Wtedy procedura przechodzi do tego dziecka typu SP, które zostało oznaczone jako udowodnione. Dla wierzchołka typu SP najpierw rekurencyjnie odtwarzane są dowody wszystkich jego dzieci, czyli poddowody przesłanek rozbicia. Następnie te poddowody są łączone w jeden ciąg linii, a na końcu dopisywana jest linia stosująca regułę zapisaną w rozbiciu.

Oczywiście łączenie dowodów wymaga odpowiedniej numeracji linii. Numeracja w każdym poddowodzie rozpoczyna się od 1, dlatego przed połączeniem kolejnych fragmentów wszystkie indeksy w danym poddowodzie są przesuwane o liczbę linii występujących we wcześniejszych fragmentach. Dotyczy to zarówno numerów samych linii, jak i odwołań do wcześniejszych linii zapisanych przy regułach.

Jeżeli mamy poddowody

$$D_1, \dots, D_n,$$

ich ostatnie linie kodują sekwenty będące przesłankami reguły, przez którą dane podproblemy zostały połączone w drzewie poszukiwania. Numery tych linii (po przenie numerowaniu) mogą więc zostać użyte jako odwołania w końcowej linii sklejającej dowód.

Kolejność tych numerów musi odpowiadać kolejności przesłanek danej reguły. W implementacji zapewnia to struktura drzewa poszukiwania: dzieci węzła reprezentującego rozbieżność są przechowywane w tej samej kolejności, w jakiej odpowiadają przesłankom reguły użytej do rozbieżności. Dzięki temu, po sklejeniu poddowodów, można odczytać numery ich końcowych linii jako uporządkowaną listę argumentów końcowej reguły.

Dla większości reguł wystarczy więc połączyć przenie numerowane poddowody, a następnie dopisać jedną linię postaci

$$k_n + 1. \text{ konkluzja rozbieżności } R, k_1, \dots, k_n,$$

gdzie $k_1, \dots, k_n$ są numerami końcowych linii odpowiednich poddowodów. Separatory występujące w uzasadnieniu linii są przy tym generowane zgodnie z formatem tekstowym danej reguły. W omawianych tu przypadkach są to przecinki oddzielające kolejne indeksy.

Niektóre reguły wymagają jednak dodatkowej ostrożności. W implementacji wyróżnione są reguły, których zapis tekstowy odwołuje się nie tylko do linii będących bezpośrednimi przesłankami reguły, ale także do dodatkowych linii pomocniczych.

Dotyczy to reguł:

$$\text{weakening}, \quad \rightarrow\text{-introduction}, \quad \neg\text{-introduction}, \quad \text{RAA}.$$

Dla takich reguł nie wystarcza wskazanie samych końcowych linii poddowodów. Trzeba jeszcze dopisać odpowiednią linię pomocniczą, do której końcowa reguła będzie mogła się odwołać. W tym celu każdą z tych linii rozpatrujemy osobno.

Przykładowo dla reguły wyprowadzania implikacji końcowa linia ma postać

$$i. \varphi \rightarrow \psi \quad \rightarrow\text{-introduction } j\text{-}k,$$

Indeks k wskazuje linię przesłanki, z kolei indeks j wskazuje linię, z której odczytujemy argument φ którego nie jesteśmy w stanie jednoznacznie zidentyfikować na podstawie samej przesłanki. W tym celu do złączonych dowodów przesłanek dodajemy linię, która przy pomocy reguły **assumption** wyprowadza argument φ . Indeks tej linii jest następnie używany jako indeks j w końcowej linii. Podobne schematy konstruujemy dla reguł **weakening**, **negation-introduction** oraz **RAA**.

Ogólny schemat sklejanie poddowodów odpowiadających jednemu rozbieżności można zapisać następująco:

Algorithm 11 Sklejanie poddowodów jednego rozbicia

```

1: procedure SKLEJ( $D_1, \dots, D_n, S$ )
2:    $R \leftarrow$  reguła zapisana w rozbiciu  $S$ 
3:    $P \leftarrow$  problem rozbijany przez  $S$ 
4:    $B \leftarrow$  pusty tekst dowodu
5:    $k \leftarrow 0$ 
6:   for  $i = 1, \dots, n$  do
7:     if  $D_i$  ma niepusty kontekst bazowy then
8:        $D_i \leftarrow$  wynik usunięcia kontekstu bazowego z  $D_i$ 
9:     end if
10:    przesun wszystkie indeksy w  $D_i$  o  $k$ 
11:    dopisz  $D_i$  do  $B$ 
12:     $k \leftarrow$  liczba linii aktualnego dowodu  $B$ 
13:  end for
14:  if  $R$  nie wymaga dodatkowej linii pomocniczej then
15:    dopisz do  $B$  linię stosującą  $R$  do końcowych linii poddowodów
16:  else
17:    dopisz odpowiednią linię pomocniczą
18:    dopisz do  $B$  linię stosującą  $R$  dla linii pomocniczej i końcowych linii
    poddowodów
19:  end if
20:  return  $B$ 
21: end procedure

```

Całe odtwarzanie dowodu z drzewa można opisać osobną procedurą. Jeżeli wierzchołek typu LP jest rozwiązaniem liściem, zwracamy zapisany w nim dowód po usunięciu kontekstu bazowego. Jeżeli wierzchołek typu LP został rozwiązany przez jedno z rozbić, przechodzimy do tego dziecka typu SP i w nim odtwarzamy dowód. Jeżeli wierzchołek jest typu SP, najpierw odtwarzamy dowody wszystkich jego dzieci, a potem sklejamy je przy użyciu rozbicia zapisanego w tym wierzchołku.

Algorithm 12 Odtwarzanie dowodu z drzewa

```

1: procedure ODTWORZDOWOD( $X$ )
2:   if  $X$  nie jest oznaczony jako udowodniony then
3:     zgłoś błąd
4:   end if
5:   if  $X$  jest liściem typu LP then
6:     return dowód zapisany w  $X$  po usunięciu kontekstu bazowego
7:   end if
8:   if  $X$  jest wewnętrznym wierzchołkiem typu LP then
9:      $Y \leftarrow$  udowodnione dziecko  $X$ 
10:    return ODTWORZDOWOD( $Y$ )
11:  end if
12:  if  $X$  jest wierzchołkiem typu SP then
13:     $L \leftarrow$  pusta lista dowodów
14:    for każde dziecko  $Y$  wierzchołka  $X$  do
15:      dopisz ODTWORZDOWOD( $Y$ ) do listy  $L$ 
16:    end for
17:    return SKLEJ( $L, X$ )
18:  end if
19: end procedure

```

W ten sposób procedura sklejania odwraca proces dekompozycji celu. Rozbijanie zastępuje jeden problem listą podproblemów, natomiast sklejanie bierze dowody tych podproblemów i przy użyciu reguły zapisanej w rozbiciu tworzy dowód problemu wyjściowego. Dzięki temu wynik działania całej strategii nie jest jedynie informacją, że teza została znaleziona w drzewie poszukiwania, ale konkretnym dowodem zapisanym w tej samej notacji tekstowej, która była używana przez parser, weryfikator i model językowy.

Rozdział 4.

Generowanie korpusu

4.1. Motywacja

Wykorzystanie strategii opisanych w poprzednim rozdziale wymaga dwóch odpowiednio wytrenowanych modeli językowych. Pierwszy z nich ma generować poprawne dowody z założeniami, w tym również dowody z pustą listą założeń, drugi zaś rozbić problemy dowodowych na podproblemy. Do ich wytrenowania potrzebne są więc duże korpusy danych zawierające przykłady takich dowodów i rozbić zapisane w przyjętej notacji tekstowej.

Naturalnym punktem wyjścia byłoby znalezienie gotowej bazy tautologii wraz z ich dowodami, a następnie przekształcenie tych dowodów do formatu używanego w niniejszej pracy. W praktyce takie podejście okazało się niewystarczające. Większość dostępnych zbiorów tautologii ma charakter dydaktyczny: są to zwykle zestawy ćwiczeń dla osób uczących się logiki, zawierające co najwyżej kilkadziesiąt przykładów. Taka liczba dowodów jest zbyt mała, aby mogła stanowić samodzielny korpus treningowy dla modelu językowego. Z tego powodu konieczne było wygenerowanie korpusu danych w sposób algorytmiczny.

Ogólny schemat generowania korpusu był następujący. Najpierw przygotowano dowody tautologii pochodzących ze skryptu do kursu *Logika dla informatyków* na Uniwersytecie Wrocławskim. Następnie dowody te połączono w jeden duży ciąg poprawnych linii dowodowych, który dalej rozszerzano przez algorytmiczne dopisywanie kolejnych losowych, ale formalnie poprawnych linii.

Z tak otrzymanego dużego ciągu wyluskiwano następnie dowody tautologii, czyli dowody linii kodujących sekweny o pustym kontekście. Otrzymane dowody były dalej przekształcane w przykłady treningowe używane w różnych częściach systemu: dowody z założeniami oraz rozbić problemy dowodowych na podproblemy. Na końcu korpusy były iteracyjnie formatowane, filtrowane i uzupełniane na podstawie wyników treningu, tak aby model możliwie dobrze uczył się generowania potrzebnych fragmentów dowodów. Szczegółowe omówienie poszczególnych etapów znajduje się

w kolejnych sekcjach tego rozdziału.

4.2. Formuły źródłowe i początkowe dowody

Pierwszy etap generowania korpusu polegał na przygotowaniu początkowego zbioru poprawnych dowodów tautologii. Zbiór ten miał pełnić rolę punktu wyjścia dla późniejszego algorytmicznego rozszerzania danych.

Na początku przygotowano listę formuł rachunku zdań pochodzących ze skryptu do kursu *Logika dla informatyków* na Uniwersytecie Wrocławskim. Część z nich była tautologiami, część formułami spełnialnymi, lecz nietautologicznymi, a część formułami sprzecznymi. Przy pomocy prostego algorytmu sprawdzania tautologiczności przez podstawianie wartości logicznych wybrano formuły tautologiczne. Do listy dodano również negacje formuł sprecznych, ponieważ one także są tautologiami.

Kolejnym etapem było przygotowanie dowodów otrzymanych tautologii. W tym celu wykorzystano zewnętrzny model językowy, któremu przekazano opis używanej notacji. Następnie ze względu na możliwość halucynacji modelu generującego dowody każdy dowód został formalnie sprawdzony przy pomocy weryfikatora omawianego w poprzednim rozdziale.

Po weryfikacji początkowe dowody zostały połączone w jeden duży ciąg linii dowodowych, który będziemy w dalszej części nazywać "dużym dowodem". Ciąg ten posłużył jako punkt wyjścia dla procedury dalszego rozszerzania korpusu.

4.3. Losowe rozszerzanie dużego dowodu

Kolejnym etapem generowania korpusu było dopisywanie nowych linii do dużego dowodu. Ogólna idea tej procedury jest prosta: najpierw losowana jest jedna z reguł wnioskowania, a następnie wybierane są wcześniejsze linie, które mogą zostać użyte jako argumenty tej reguły. Jeżeli dana reguła wymaga dodatkowego argumentu, który nie wynika jednoznacznie z wybranych przesłanek, argument ten jest generowany osobno. Przykładowo dla linii postaci

i. φ assumption

konieczne jest wygenerowanie formuły φ .

Najprostszą metodą tej procedury polegałoby na losowym wybieraniu potencjalnych argumentów reguły, konstruowaniu kandydackiej linii, a następnie sprawdzaniu, czy dowód rozszerzony o tę linię poprawnie się parsuje. Jeśli parsowanie kończyło się sukcesem, linia była dopisywana do dużego dowodu; w przeciwnym razie kandydataura była odrzucana.

W praktyce takie podejście opierające się na tej idei okazało się jednak zbyt wolne. Dla części reguł już przy stosunkowo niewielkim rozmiarze dużego dowodu prawdopodobieństwo wylosowania pasujących argumentów było małe. W efekcie znalezienie jednej poprawnej kandydackiej linii mogło wymagać bardzo dużej liczby prób, co znacząco spowalniało cały proces generowania danych.

Możliwym rozwiązaniem byłoby wprowadzić maksymalną liczbę prób dla danej reguły i po jej przekroczeniu porzucać próbę dopisania linii, przechodząc do losowania następnej reguły. Takie rozwiązanie prowadziłoby jednak do niekorzystnego rozkładu przykładów w korpusie. Reguły trudniejsze do zastosowania, wymagające spełnienia większej liczby warunków, pojawiałyby się znacznie rzadziej, mimo że często są istotne w rzeczywistych dowodach. Jednocześnie nadmiernie reprezentowane byłyby reguły bardzo proste, takie jak **assumption**, czy szczególnie trywialne w procesie generacji ale rzadko potrzebne w praktyce, takie jak $\top$ -**introduction**.

Prowadziłoby to do sytuacji szczególnie niekorzystnej z punktu widzenia treningu modelu: reguły trudne do nauczenia i ważne dla konstrukcji dowodów byłyby słabo reprezentowane w danych, natomiast reguły proste lub mało informacyjne dominowałyby w korpusie. Z tego powodu konieczne było zoptymalizowanie samego procesu doboru argumentów dla losowanych reguł.

Naturalnym kierunkiem optymalizacji jest zawężenie przestrzeni przeszukiwań, by zwiększyć prawdopodobieństwo wylosowania pasujących argumentów. Np dla reguły

$$\begin{array}{l}
 \text{j. } \phi \quad \dots \\
 \dots \\
 \text{k. } \phi \rightarrow \psi \quad \dots \\
 \dots \\
 \text{i. } \psi \quad \rightarrow\text{-elimination, k, j}
 \end{array}$$

której zapis w notacji drzewiastej ma postać

$$\frac{\Gamma_1 \vdash \phi \rightarrow \psi \quad \Gamma_2 \vdash \phi}{\Gamma_1, \Gamma_2 \vdash \psi} (\rightarrow E)$$

chcielibyśmy na początku ograniczyć się do losowania linii, których konkluzja jest implikacją, a następnie do linii, której konkluzja jest lewą stroną tej implikacji.

W tym celu linie dużego dowodu są grupowane w kilku słownikach. Pierwszy słownik grupuje indeksy linii według typu tezy sekwentu. Kluczami są tu rodzaje formuł, np. koniunkcja, alternatywa, negacja, szczególny przypadek podwójnej negacji czy formuła atomowa, z kolei wartościami są indeksy linii danego typu. Drugi słownik grupuje indeksy według dokładnej postaci tezy. Kluczami są tu konkretne formuły, a wartościami indeksy linii, których tezą jest dana formuła. Trzeci słownik grupuje indeksy linii według założeń występujących w kontekście. Kluczami są tu

formuły, a wartościami indeksy linii, których kontekst zawiera daną formułę. Ostatnią pomocniczą strukturą jest zestaw indeksów linii, które mogą być użyte jako przesłanki reguły

$$\frac{\Gamma, \neg\varphi \vdash \perp}{\Gamma \vdash \varphi} \text{ (RAA)}$$

czyli takich, których tezą jest $\perp$ i które mają w kontekście formułę będącą negacją.

Słowniki przechowujemy w dwóch wariantach: pierwszy mapuje klucze na listy odpowiednich indeksów, drugi z kolei na zbiory. Służy to optymalizacji różnych typów operacji. Listy są wygodne do losowania, natomiast zbiory do szybkiego sprawdzania przynależności i wyznaczania przecięć.

Dopisanie linii dla każdej reguły wnioskowania jest w implementacji obsługiwane osobno, ponieważ poszczególne reguły wymagają innych warunków doboru przesłanek oraz, w niektórych przypadkach, dodatkowych argumentów. Szczegółowe omawianie wszystkich tych przypadków nie jest jednak konieczne do zrozumienia ogólnej idei generatora. Aby pokazać sposób wykorzystania opisanych słowników, przyjrzymy się regule eliminacji alternatywy, która w notacji drzewiastej ma postać

$$\frac{\Gamma_1 \vdash \varphi \vee \psi \quad \Gamma_2, \varphi \vdash \rho \quad \Gamma_3, \psi \vdash \rho}{\Gamma_1, \Gamma_2, \Gamma_3 \vdash \rho} (\vee E).$$

zaś w notacji tekstowej ma postać

```
j.   $\phi \vee \psi$   ...
...
k.   $\rho$       ...
...
l.   $\rho$       ...
...
i.   $\rho$        $\vee$ -elimination, j, k, l
```

Aby dopisać linię przez zastosowanie tej reguły, generator musi znaleźć trzy linie o indeksach j, k i l spełniające określone warunki: konkluzją pierwszej z nich powinna być alternatywa, druga powinna mieć w kontekście lewą stronę tej alternatywy, trzecia powinna mieć w kontekście jej prawą stronę, a konkluzje drugiej i trzeciej linii powinny być identyczne.

Bez omawianych słowników znalezienie takiej trójki wymagałoby bardzo wielu prób. Liczba możliwych trójek rośnie sześciennie względem liczby linii w dużym dowodzie, a jedynie nieliczne z nich spełniają wszystkie wymagane warunki. Użycie słowników pozwala znacząco ograniczyć przestrzeń przeszukiwania. Procedura znajdowania odpowiednich przesłanek wygląda następująco.

Najpierw losowana jest linia z listy linii, których tezą jest alternatywa. Oznaczmy tę tezę jako $\varphi \vee \psi$. Następnie tworzone są dwa zbiory: zbiór tez linii mających w kontekście φ oraz zbiór tez linii mających w kontekście ψ . Jeżeli przekrój tych zbiorów jest niepusty, losowana jest z niego formuła ρ . W przeciwnym wypadku procedura wraca do wyboru pierwszej przesłanki, czyli linii z alternatywą.

Po wybraniu formuły ρ druga przesłanka losowana jest z przekroju zbioru linii mających w kontekście φ oraz zbioru linii, których tezą jest ρ . Analogicznie trzecia przesłanka losowana jest z przekroju zbioru linii mających w kontekście ψ oraz zbioru linii, których tezą jest ρ . W ten sposób otrzymujemy trójkę linii potrzebnych do zastosowania reguły eliminacji alternatywy.

Słowniki pomocnicze oraz sparsowana reprezentacja dużego dowodu są aktualizowane inkrementalnie. Po dopisaniu kolejnej poprawnej linii system nie parsuje ponownie całego dowodu ani nie buduje od zera struktur pomocniczych. Zamiast tego korzysta z reprezentacji uzyskanej w poprzedniej iteracji i uzupełnia ją o informacje wynikające z nowej linii: jej sparsowaną postać oraz indeksy potrzebne w odpowiednich słownikach.

Koszt tej operacji nie musi być formalnie stały, ponieważ zależy od użytych kontenerów i od liczby miejsc, w których należy zapisać informację o nowej linii. Jest jednak kosztem lokalnej aktualizacji, a nie ponownego przetwarzania całego dowodu, dlatego w praktyce rośnie znacznie wolniej niż koszt każdorazowego odtwarzania wszystkich struktur pomocniczych.

Mimo zastosowanych usprawnień, przy dużych dowodach rzędu kilkudziesięciu tysięcy linii czas potrzebny na dopisanie kolejnej linii nadal zauważalnie rośnie. Z tego powodu procedura generacji uruchamiana jest wielokrotnie i równolegle, w kilku wątkach. Wszystkie wątki startują od tych samych początkowych dużych dowodów, jednak każdy z nich pracuje na własnej kopii dowodu, rozwijając ją niezależnie od pozostałych. Każdego dowodu generacji dla jednego dużego dowodu generuje kilkanaście tysięcy liniiz.

4.4. Generowanie z dużego dowodu przykładów treningowych

Same duże dowody nie są bezpośrednio użyteczne jako dane treningowe dla modeli językowych. Wynika to z kilku powodów.

Po pierwsze, ich format różni się od formatu przykładów, które model ma później generować. W przypadku rozbić celu format rozbić i dużego dowodu jest całkowicie różny, w przypadku dowodów z założeniami w dużych dowodach brakuje linii kodujących cel oraz linii kodujących założenia.

Po drugie, linie dużego dowodu powstają w sposób losowy. Oznacza to, że cho-

ciaż cały ciąg jest formalnie poprawny, to nie stanowi uporządkowanej odpowiedzi na konkretny problem dowodowy. Nie zawiera więc bezpośrednio informacji o tym, jakich reguł i argumentów należy używać, aby rozwiązać zadany sekwent.

Po trzecie, duże dowody są zbyt długie, aby mogły być w całości przetwarzane przez model językowy. Ich długość znacząco przekracza długość kontekstu obsługiwaną przez wytrenowane modele. W rezultacie model, widząc późniejszą linię dowodu, nie miałby już dostępu do wielu wcześniejszych linii, do których ta linia się odwołuje. Uczenie na takich przykładach nie pozwalałoby więc skutecznie nauczyć się doboru argumentów dla poszczególnych reguł.

Z tego powodu duże dowody musiały zostać przekształcone do postaci użytecznej dla treningu. Proces ten można podzielić na trzy zasadnicze etapy:

1. **Wyłuskiwanie dowodów tautologii.** Z dużego dowodu wybierane są linie kodujące sekwenty o pustym kontekście. Każda taka linia odpowiada dowodowi pewnej tautologii. Następnie odtwarzany jest fragment dużego dowodu potrzebny do uzasadnienia tej linii, a otrzymane dowody podlegają dalszej selekcji.
2. **Wyłuskiwanie dowodów z założeniami z dowodów bez założeń i formatowanie powstałego w ten sposób korpusu** Z otrzymanych dowodów tautologii dla ich poszczególnych linii generowane są dowody z założeniami oraz rozbicia.
3. **Wyłuskiwanie rozbić problemów na podproblemy z dowodów bez założeń i formatowanie powstałego w ten sposób korpusu**

4.4.1. Wyłuskiwanie dowodów tautologii

Pierwszym etapem wyłuskiwania dowodów tautologii jest sparsowanie dużego dowodu. Następnie znajdujemy wszystkie linie, których sekwenty po sparsowaniu mają puste konteksty (formuły zapisane w tych liniach są tautologiami).

Następnie tworzymy zbiór linii użytych w dowodzie tej tautologii. początkowo zbiór ten zawiera tylko linię, którą chcemy udowodnić, a następnie dodajemy do tego zbioru wszystkie linie, do których odwołuje się ta linia. Potem analogicznie dodajemy linie, do których odwołują się nowo dodane linie, i powtarzamy ten proces, aż otrzymany zbiór linii będzie zamknięty ze względu na odwołania.

Na końcu wybrane linie sortujemy zgodnie z ich kolejnością w dużym dowodzie, łączymy je w jeden dowód i przenumeryujemy je tak, aby indeksy kolejnych linii były kolejnymi liczbami naturalnymi, zaczynając od 1.

W tej fazie dokonujemy również wstępnej selekcji dowodów które nas interesują.

Odrzucamy:

1. **Większość dowodów mających mniej niż 3 linie (zostawiamy losowo około 0.01 ich początkowej liczby).** W szczególności odrzucamy większość jednolinijkowych "dowodów" tautologii przy użyciu reguł TND i T-introduction.
2. **Dowody tautologii zawierających symbole $\perp$ i $\top$ (zostawiamy losowo około 0.01 ich początkowej liczby).** Tautologie te początkowo miały dość dużą nadreprezentację, a ich dowody często sprowadzały się do mechanicznego wykorzystania własności stałych logicznych, przez co wносиły niewiele informacji o ogólnych schematach dowodzenia.
3. **Wszystkie dwody mające więcej niż 1024 znaki.** Odrzucono je, ponieważ rozmiar kontekstu trenowanego modelu językowego wynosił 1024

W wygenerowanym zbiorze pojawiało się wiele dowodów tych samych tautologii lub tautologii różniących się jedynie nazwami zmiennych. Aby usunąć tego typu powtórzenia, dowody grupowano ze względu na znormalizowaną postać dowodzonej tautologii. W postaci znormalizowanej pierwsza zmienna występująca w formule od lewej była zastępowana przez x_1 , druga przez x_2 , trzecia przez x_3 itd. Następnie dla każdej grupy wybierano dowód o najmniejszej liczbie linii, a pozostałe dowody odrzucano ze zbioru z którego będziemy generować korpusy uczące.

4.4.2. Tworzenie dowodów z założeniami

Z otrzymanych dowodów tautologii generuję następnie przykłady dowodów sekwentów z założeniami. Każda linia poprawnego dowodu koduje pewien sekwent $\Gamma \vdash \varphi$, a poprzedzające ją linie zawierają jego dowód. Będziemy więc odtwarzać dowód φ w kontekście Γ .

Dla każdej linii dowodu odtwarzano najpierw fragment potrzebny do jej uzasadnienia. Procedura ta przypomina procedurę ekstrakcji dowodów tautologii z dużego dowodu. Zaczynała ona od wybranej linii, a następnie dodawała wszystkie linie, do których odwołuje się jej uzasadnienie. Następnie analogicznie dodawano linie, do których odwołują się nowo dodane linie. Proces powtarzano aż do otrzymania zbioru linii zamkniętego ze względu na odwołania. W ten sposób z całego dowodu wycinano poddowód wystarczający do uzasadnienia wybranego sekwentu. Po wyborze odpowiednie linie były ze sobą sklejane, a ich numeracja oraz numeracja odwołań była zmieniana tak, aby numery kolejnych linii były kolejnymi liczbami naturalnymi zaczynając od 1.

Następnie tworzono zapytanie zgodne z formatem dowodów z kontekstem, używanym w korpusie. Zapytanie rozpoczyna się linią, w której po **Prove that:** zapisywana jest formuła, którą chcemy udowodnić. Jeżeli zbiór założeń jest niepusty, to w kolejnej linii po tekście **assuming that:** zapisywane jest pierwsze założenie, a kolejne założenia są zapisywane po przecinku w następnych liniach.

Po złączeniu części odpowiadającej za definicję problemu i części zawierającej dowód otrzymujemy dowód w formacie, który mógłby zostać użyty w korpusie. Niestety taki dowód wciąż może zawierać liczne nadmiarowe linie, co utrudniałoby modelowi zrozumienie, które kroki faktycznie zbliżają nas do udowodnienia sekwentu. W tym celu stosujemy kilka uproszczeń.

Naturalnym pierwszym kierunkiem uproszczeń było skorzystanie z jawnego kontekstu problemu. Jeżeli w dowodzie pojawiała się linia dowodząca formuły należącej do tego kontekstu, zastępowano jej uzasadnienie odwołaniem do kontekstu. W zależności od tego, czy linia miała pusty czy niepusty kontekst dodatkowy, używano reguły **context** dla argumentu 0 albo reguły **contextWeak**, również dla argumentu 0. Linia 0 została zdefiniowana wcześniej jako niezapisana wprost linia, której kontekst jest równy kontekstowi bazowemu. Po tych podmianach ponownie usuwano linie, które przestały być potrzebne, a cały fragment przenumerowywano. W rezultacie powstawał krótszy i bardziej regularny dowód z założeniami, lepiej nadający się do treningu modelu.

Może się również zdarzyć, że podczas dowodzenia sekwentu z założeniami jakaś linia dowodu, która nie jest ostatnią linią, także koduje sekwent, który chcemy udowodnić. Ponieważ dowód może zakończyć się na pierwszej linii kodującej żądany sekwent, usuwano wszystkie linie znajdujące się po pierwszym wystąpieniu tego sekwentu.

Kolejne uproszczenie bazuje na następującej intuicji. Zdefiniujemy indukcyjnie zbiór linii ważnych. Niech n oznacza numer ostatniej linii dowodu P , zaś $\text{Ref}(k, P)$ oznacza zbiór linii, do których odwołuje się linia k w dowodzie P . Przyjmujemy

$$W_{1,P} = \{n\}.$$

Następnie definiujemy

$$W_{i+1,P} = \bigcup_{k \in W_{i,P}} \text{Ref}(k, P).$$

Ostatecznie niech

$$W_P = \bigcup_{i \in \mathbb{N}} W_{i,P}.$$

Zbiór W_P zawiera więc ostatnią linię dowodu, linie, do których ona się odwołuje, linie, do których odwołują się te linie itd. Innymi słowy są to linie, które są potrzebne do uzasadnienia końcowego sekwentu.

Zauważmy, że dowód z założeniami P pozostanie poprawnym dowodem żadanego sekwentu, jeżeli usuniemy z niego wszystkie linie nienależące do W_P , a w pozostałych liniach zmienimy numerację linii i odwołań tak, aby numery kolejnych linii były kolejnymi liczbami naturalnymi. Linie spoza W_P nie są bowiem wykorzystywane w uzasadnieniu ostatniej linii dowodu.

Konkluzje reguł są często w pewnym sensie podobne do ich przesłanek. Na przykład mogą mieć takie same konteksty, konteksty różniące się jedną formułą, albo

w przypadku reguły **weakening** tę samą konkluzję. Można więc przypuszczać, że w niektórych miejscach dowodu da się zmienić odwołanie do danej linii na odwołanie do innej, wcześniejszej linii, nie niszcząc poprawności dowodu. W ten sposób, zmieniając numery odwołań, możemy wpływać na rozmiar zbioru W_P . Jeżeli po takiej zmianie część późniejszych linii przestanie należeć do W_P , będzie można usunąć je z dowodu.

Znalezienie dokładnego algorytmu, który wybierałby odwołania prowadzące do najmniejszego możliwego zbioru W_P , wydaje się trudne. Dlatego zastosowano prostą heurystykę: jeżeli w każdym odwołaniu będziemy starali się odwoływać do jak najwcześniejszej linii, to powinno to sprzyjać zmniejszaniu liczby późnych linii należących do W_P . W szczególności późniejsze linie często są wynikiem wielu wcześniejszych kroków dowodowych, z których każdy posiada własne odwołania. Obecność takiej linii w W_P może więc pociągać za sobą obecność wielu innych linii.

Aby ująć to dokładniej, dla linii k zdefiniujemy

$$\text{Ref}_1(k, P) = \text{Ref}(k, P),$$

$$\text{Ref}_{i+1}(k, P) = \bigcup_{j \in \text{Ref}_i(k, P)} \text{Ref}(j, P),$$

oraz

$$\overline{\text{Ref}}(k, P) = \bigcup_{i \in \mathbb{N}} \text{Ref}_i(k, P).$$

Zbiór $\overline{\text{Ref}}(k, P)$ zawiera więc linie, od których linia k zależy pośrednio i bezpośrednio. Intuicyjnie, im większy jest indeks k , tym większa jest szansa, że $\overline{\text{Ref}}(k, P)$ będzie duży. Jeżeli taka późna linia należy do W_P , to razem z nią w W_P znajdują się również linie należące do $\overline{\text{Ref}}(k, P)$. Jest to kolejny powód, dla którego preferowane są odwołania do linii pojawiających się w dowodzie możliwie wcześnie.

Sama procedura zmian odwołań, ze względu na niewielkie rozmiary treningowych dowodów, jest dość prosta. Dla każdego odwołania w dowodzie sprawdzamy, czy zmiana wskazywanego indeksu na kolejne liczby naturalne zaczynając od 0 nie psuje poprawności dowodu. Jeżeli po takiej zmianie dowód nadal przechodzi weryfikację, podmiana jest zachowywana. Jako poprawność rozumiemy tutaj zarówno to, że dowód pozostaje ciągiem poprawnych kroków, jak i to, że nadal dowodzi żadanego sekwentu. Po dokonaniu wszystkich takich zmian (początkowo zmieniamy odwołania w ostatniej linii, następnie idziemy w górę) z dowodu usuwane są linie nienależące do W_P , a następnie zmieniana jest numeracja linii i odwołań tak, aby indeksy linii były kolejnymi liczbami naturalnymi zaczynając od 1.

4.4.3. Tworzenie rozbić sekwentów

Podobnie jak w przypadku generacji dowodów z założeniami, rozbicia generujemy z poszczególnych linii dowodu bez założeń.

Rozbicie składa się z problemu głównego, reguły użytej do jego rozbicia oraz listy podproblemów odpowiadających przesłankom tej reguły. Jeżeli dana linia dowodu koduje sekwent P i została uzyskana za pomocą reguły R z linii kodujących przesłanki $P_1, \dots, P_n$, to możemy odczytać z niej rozbicie problemu P na podproblemy $P_1, \dots, P_n$ za pomocą reguły R .

Jak zdefiniowano w sekcji ??, sparsowany dowód jest słownikiem mapującym indeksy linii na odpowiadające im sparsowane linie. Każda sparsowana linia zawiera sekwent zapisany w danej linii, regułę użytą do jej uzasadnienia oraz indeksy linii, do których odwołuje się to uzasadnienie. Mając więc sparsowany dowód oraz wybraną linię tego dowodu, możemy pobrać problem główny i regułę bezpośrednio z tej linii, natomiast podproblemy otrzymać przez odwołanie się do wcześniejszych linii wskazanych przez jej argumenty.

Należy jednak zaznaczyć, że nie każdy indeks występujący w zapisie uzasadnienia linii odpowiada rzeczywistej przesłance rozbicia. Jak wyjaśnialiśmy wcześniej w przypadku niektórych reguł część indeksów odwołuje się do linii które nie kodują przesłanek ale które są konieczne, żeby odczytać jakiś argument reguły który nie wynika bezpośrednio z przesłanek.

Następnie rozbicia zostały przekonwertowane do formy tekstowej gdzie pierwsza linia składa się z napisu **Smash:** po którym następuje zapis sekwentu który rozbijamy, druga linia składa się z napisu **With:** po którym następuje nazwa reguły używanej do rozbicia, a zaś trzecia napis **To:** po którym jeżeli lista podproblemów na które rozbijamy sekwent jest niepusta jest zapisana pierwsza przesłanka, z kolei kolejne są zapisywane po przecinku w następnych liniach.

Wadą wygenerowanego korpusu rozbić było to, że konkluzją zdecydowanej większości rozbić przy pomocy reguły

$$\frac{\Gamma_1, \varphi \vdash \psi \quad \Gamma_2, \psi \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \varphi \leftrightarrow \psi} (\leftrightarrow I)$$

formuły φ i ψ były identyczne, co przy praktycznym szukaniu dowodów sekwentów logicznych zdarza się rzadko. W tym celu dodałem do korpusu rozbicia przy pomocy reguły wyprowadzenia równoważności wygenerowane przez osobny algorytm.

Wartościowaniem zbioru zmiennych nazwijmy funkcję, która zmiennym z tego zbioru przypisuje wartości $\top, \perp$. Dla zbioru n zmiennych istnieje 2^n wartościowań.

Przyjmijmy, że mamy skończony n -elementowy zestaw zmiennych A , które będą występować w rozważanych formułach i kontekstach.

Jako

$$\text{Sat}(X)$$

zdefiniujemy zbiór wszystkich wartościowań zmiennych z A , które spełniają X jeśli X jest formułą i zbiór wszystkich wartościowań zmiennych z A , które spełniają

wszystkie formuły w X jeśli X jest kontekstem.

Zauważmy, że

$$\text{Sat}(\Gamma, \varphi) = \text{Sat}(\Gamma) \cap \text{Sat}(\varphi)$$

Istotnie, jeżeli jakieś wartościowanie należy do $\text{Sat}(\Gamma, \varphi)$ to musi spełniać zarówno formułę φ jak i jak i formuły w Γ czyli należeć zarówno do $\text{Sat}(\varphi)$ jak i do $\text{Sat}(\Gamma)$, czyli należeć do przecięcia tych zbiorów. W drugą stronę, jeżeli wartościowanie należy do $\text{Sat}(\Gamma) \cap \text{Sat}(\varphi)$ to spełnia zarówno formułę φ jak i formuły z Γ , czyli należy do $\text{Sat}(\Gamma, \varphi)$.

Zauważmy również, że

$$\Gamma \vdash \phi \leftrightarrow \text{Sat}(\Gamma) \subseteq \text{Sat}(\phi)$$

Istotnie zapis $\Gamma \vdash \phi$ oznacza, że z formuł z Γ da się dowieść ϕ , co w przypadku rachunku zdań w logice klasycznej jest równoważne temu, że jeżeli dla jakiegoś prawdziwe są wszystkie formuły z Γ to dlatego wartościowania prawdziwa jest formuła ϕ , co jest równoważne z tym, że jeżeli jakieś wartościowanie należy do $\text{Sat}(\Gamma)$ musi należeć do $\text{Sat}(\phi)$, czyli, że $\text{Sat}(\Gamma)$ zawiera się w $\text{Sat}(\phi)$.

Algorytm wygenerowania rozbicia przy pomocy

$$\frac{\Gamma_1, \varphi \vdash \psi \quad \Gamma_2, \psi \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \varphi \leftrightarrow \psi} (\leftrightarrow I)$$

będzie polegał, na znalezieniu przesłanek tego rozbicia, by później wywnioskować z nich konkluzję, a potem złączyć konkluzję, przesłanki i rozbicie w sparsowane rozbicie, które później sprowadzimy do postaci tekstowej.

Żeby znaleźć przesłanki reguły musimy znaleźć konteksty Γ_1 i Γ_2 oraz formuły φ, ψ takie, że

$$\Gamma_1, \varphi \vdash \psi$$

oraz

$$\Gamma_2, \psi \vdash \varphi$$

Ze wcześniejszych obserwacji wynika, że warunki te są równoważne z

$$\text{Sat}(\Gamma_1) \cap \text{Sat}(\varphi) \subseteq \text{Sat}(\psi)$$

oraz

$$\text{Sat}(\Gamma_2) \cap \text{Sat}(\psi) \subseteq \text{Sat}(\varphi)$$

Na początku nasz algorytm losuje konteksty Γ_1 i Γ_2 , a następnie w pętli losuje parę formuł φ i ψ tak aby spełniały powyższe warunki. Taka para zawsze istnieje dla dowolnych kontekstów Γ_1 i Γ_2 np zawsze warunki te spełnia para tautologii, lub formuł sprzecznych. Samo losowanie formuł wydaje się operacją, która wymaga bardzo wielu powtórzeń, jednak ograniczając liczbę różnych zmiennych które mogą

występować łącznie w generowanych omawianym algorytmem kontekstach i formułach do 6 udaje się wygenerować przy pomocy opisywanej metody wiele rozbić w stosunkowo krótkim czasie.

Kolejnym problemem jest to, że nie zawsze wymagane właściwości przesłanek i konkluzji w wygenerowanych na tym etapie danych przekładają się na proste tekstowe wzorce w łatwo identyfikowane przez modele językowe. Na przykład weźmy regułę

$$\frac{\Gamma_1 \vdash \varphi \vee \psi \quad \Gamma_2, \varphi \vdash \rho \quad \Gamma_3, \psi \vdash \rho}{\Gamma_1, \Gamma_2, \Gamma_3 \vdash \rho} (\vee E)$$

Przykład jej użycia w notacji drzewiastej mógłby wyglądać następująco

$$\frac{a \wedge h \vdash a \vee b \quad a, c \wedge d, f \vdash c \quad c, b, e \vdash c}{c \wedge d, e, c, f, a \wedge h \vdash c} (\vee E)$$

co w notacji zapisu rozbić przekładało by się na następujący tekst:

```
Smash: c ∧ d, e, c, f, a ⊢ c
With: DisjunctionElimination
To: a ∧ h ⊢ a ∨ b,
    a, c ∧ d, f ⊢ c,
    c, b, e ⊢ c
```

Z podanego napisu trudno wywnioskować na pierwszy rzut oka jakikolwiek regularność. Jednocześnie zauważmy, że przy generacji rozbitcia przy pomocy eliminacji alternatywy, generujemy również poprawne rozbitcia jeśli założymy, że $\Gamma_1, \Gamma_2, \Gamma_3$ są takie same i równe ich sumie a wtedy jeżeli zachowamy odpowiednią kolejność wypisywania przesłanek sekwentów (co zapewniamy odpowiednią implementacją działań na kontekstach) możemy zamienić powyższy zapis rozbitcia na

```
Smash: c ∧ d, e, c, f, a ∧ h ⊢ c
With: DisjunctionElimination
To: c ∧ d, e, c, f, a ∧ h ⊢ a ∨ b
    c ∧ d, e, c, f, a ∧ h, a ⊢ c,
    c ∧ d, e, c, f, a ∧ h, b ⊢ c
```

gdzie wzorzec jest dużo bardziej zauważalny. Konteksty konkluzji i przesłanek są niemal identyczne i w przypadku dwóch ostatnich przesłanek różnią się jedynie jedną formułą na końcu.

Aby możliwie uprościć modelowi znajdowanie porządkanych wzorców nalogiczne przekształcenia, których celem jest możliwie regularny zapis rozbitcia definiujemy dla rozbić przy pomocy pozostałych reguł. Transformacje te polegają, na dodaniu do kontekstów przesłanek rozbitcia pewne, być może puste, zbiory formuł i ustawieniu formuł w kontekstach w odpowiedniej kolejności. Transformacja to odbywa się

w taki sposób, aby jeżeli początkowe rozbiecie było poprawne (to znaczy pasowało do schematu reguły i miało w przesłankach sekweny tautologiczne) otrzymane po transformacji rozbiecie nadal było poprawne a także miało tą samą regułę i konkluzję co przed transformacją. Transformacje te stosujemy do wszystkich rozbić w korpusie.

Ostatnim etapem formatowania korpusu rozbić jest modyfikacja częstości występowania rozbić tworzonych przez poszczególne reguły. W korpusie który wygenerowaliśmy do tej pory częstość występowania rozbić generowanych przez niektóre reguły znacznie przewyższała częstości innych, przez co model często używał częściej występujących reguł bez nawet w sytuacjach w których nie były one użyteczne. Co więcej ideą rozbić jest to, by rozbijały problemy na podproblemy prostsze, a dobrą heurystyką na to czy dany problem jest prostszy czy trudniejszy jest długość jego konkluzji. Niektóre reguły rozbijają swoją konkluzję na przesłanki których konkluzje są dłuższe niż konkluzja konkluzji reguły (chcielibyśmy mieć ich mniej w korpusie) na przykład

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi} (\vee I_1)$$

inne zaś rozbijają konkluzję reguły na przesłanki o konkluzjach krótszych niż konkluzje konkluzji reguły (chcielibyśmy mieć ich więcej w korpusie) na przykład

$$\frac{\Gamma_1 \vdash \varphi \quad \Gamma_2 \vdash \psi}{\Gamma_1, \Gamma_2 \vdash \varphi \wedge \psi} (\wedge I)$$

W tym celu rozbiecia w korpusie grupujemy ze względu na reguły w nich występujące, a następnie wielokrotnie losujemy z intuicyjnie ustalonym rozkładem regułę generującą rozbiecie, a następnie jednostajnie losujemy rozbiecie ze zbioru rozbić generowanych przez tą regułę i dodajemy je do nowego korpusu. W ten sposób powstaje ostateczna wersja danych treningowych dla modelu odpowiadającego za rozbiecia.

- 4.5. Losowe rozszerzanie dużego dowodu
- 4.6. Wyłuskiwanie dowodów tautologii
- 4.7. Normalizacja formuł i usuwanie powtórzeń
- 4.8. Generowanie problemów pomocniczych
- 4.9. Generowanie smashy
- 4.10. Normalizacja kontekstów w smashach
- 4.11. Równoważność jako przypadek specjalny
- 4.12. Walidacja i czyszczenie korpusu
- 4.13. Podsumowanie

Rozdział 5.

Trenowanie i korzystanie z
modelu (NanoGPTBundle i te
wszystkie funkcje z nim
związane !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
podkreślam
WSZYSTKIE(liczenie
prawdopodobieństwa,
dodawanie tokenu, linii,
trenowanie, dotrenowanie,
ładowanie)!!!!)